CarHawk Ultimate

Complete Specification Pack

QUANTUM-2.0.0

~21,400 lines across 64 files

Google Apps Script V8 Runtime

Table of Contents

CarHawk Ultimate — System Overview & Architecture

Spec Pack Part 1 of 9 | QUANTUM-2.0.0

A) System Identity

Property	Value
Name	CarHawk Ultimate CRM
Version	QUANTUM-2.0.0
Signature	■■■
Runtime	Google Apps Script V8
Timezone	America/Chicago
Home Base	St. Louis, MO (38.6270, -90.1994)
Home ZIP	63101
AI Engine	OpenAI GPT-4 Turbo Preview
Support	quantumsupport@carhawkultra.com

B) What It Does

CarHawk Ultimate is an enterprise-grade Google Apps Script suite for AI-powered vehicle deal analysis, CRM automation, rental fleet management, marketing campaigns, and pipeline intelligence. It turns a Google Sheet into a full deal-sourcing and CRM platform for car flippers and small dealers.

Core Workflow:

1. **Source** -- Browse.AI robots scrape 10+ marketplaces automatically
2. **Import** -- Raw listings flow into Master Import, get parsed and enriched
3. **Analyze** -- OpenAI GPT-4 scores every deal across 15+ dimensions
4. **Verdict** -- Each deal gets a verdict: HOT DEAL / SOLID DEAL / PORTFOLIO FOUNDATION / PASS
5. **Engage** -- Automated SMS/Email follow-up sequences contact sellers
6. **Track** -- Full CRM pipeline: leads, appointments, calls, campaigns, closed deals
7. **Decide** -- Flip vs. Hold (Turo) decision framework with financial modeling
8. **Arbitrage** -- Project-based sourcing: define needs, auto-match listings, AI-evaluate fits (PAM)
9. **Report** -- Dashboards, weekly/monthly reports, ROI optimization

C) File Inventory

Total Codebase: ~21,400 lines across 64 files

Quantum CRM Core (31 .gs files)

#	File	Lines	Purpose
1	quantum_config.gs	24	Central version & identity constants

#	File	Lines	Purpose
2	quantum_core.gs	110	State management, sheet definitions, config, capital tiers
3	quantum_utilities.gs	392	Settings, logging, ID generation, data helpers
4	quantum_headers.gs	286	Header deployment for all 20 sheets (444+ columns)
5	quantum_setup.gs	97	System initialization and 8-phase deployment
6	quantum_menu.gs	125	Main menu with 9 submenus (63+ items)
7	quantum_menu_handlers.gs	538	All menu action handlers and UI launchers
8	quantum_formulas.gs	18	Formula deployment (placeholder)
9	quantum_fallback.gs	92	AI fallback analysis, verdict logging, model init
10	quantum_import.gs	604	Browse.AI import pipeline and vehicle parsing
11	quantum_ai.gs	299	OpenAI-powered deal analysis engine
12	quantum_calculations.gs	371	ROI, MAO, scoring, market advantage
13	quantum_browse_ai.gs	421	Browse.AI sheet-based robot integration
14	quantum_browse_ai_api.gs	979	Browse.AI v2 API: tasks, bulk runs, webhooks
15	quantum_robots.gs	1584	Master robot registry for 10 marketplace platforms
16	quantum_sms_it.gs	175	SMS-iT campaign export and messaging
17	quantum_ohmylead.gs	83	Ohmylead appointment sync
18	quantum_companyhub.gs	237	CompanyHub CRM export with CSV generation
19	quantum_crm_engine.gs	379	CRM core: appointments, follow-ups, campaigns, SMS, calls
20	quantum_crm_helpers.gs	220	Intent analysis, sentiment, template filling, deal lookup
21	quantum_crm_automation.gs	143	Trigger-based follow-up, campaign, reminder processing
22	quantum_crm_api.gs	139	Twilio SMS, SendGrid email, SMS-iT API
23	quantum_knowledge_base.gs	65	Vehicle knowledge base with 10 model profiles
24	quantum_integrations.gs	93	Integration CRUD and sync tracking
25	quantum_alerts.gs	272	Real-time alerts, email notifications, digest
26	quantum_ui.gs	224	Deal gallery, quick actions, deal analyzer

#	File	Lines	Purpose
27	quantum_dashboard.gs	202	Dashboard generation with metrics and leaderboard
28	quantum_testing.gs	152	CRM test functions and simulation
29	quantum_reports.gs	196	Closed deals, weekly, monthly reports, ROI optimizer
30	quantum_triggers.gs	101	Time-based automation (hourly sync, daily analysis)
31	quantum_setup_html.gs	309	Setup wizard HTML (glassmorphism UI)
	quantum_processing_html.gs	87	Import progress dialog
	quantum_sms_export_html.gs	322	SMS export campaign builder dialog

Turo Rental Module (7 .gs files -- see TURO_SPEC_PACK.md)

#	File	Lines	Purpose
1	turo_config.gs	563	Vehicle classification, pricing, seasonality
2	turo_setup.gs	603	Idempotent sheet creation for 5 Turo sheets
3	turo_engine.gs	872	Turo economics, hold score, risk tiers
4	turo_fleet.gs	448	Fleet lifecycle management and ROI tracking
5	turo_maintenance.gs	239	Maintenance event logging with HTML dialog
6	turo_compliance.gs	235	Insurance, registration, inspection alerts
7	turo_tests.gs	565	7-test acceptance suite

Project Arbitrage Module (8 files -- see SPEC_PACK_PART9_PAM.md)

#	File	Lines	Purpose
1	PAM_Settings.gs	113	Settings sheet management and defaults
2	PAM_Utils.gs	154	ID generation, keyword scoring, formatting helpers
3	PAM_Sheets.gs	258	Idempotent sheet creation for 6 PAM sheets
4	PAM_DataAccess.gs	310	CRUD operations for all PAM sheets
5	PAM_LogicEngine.gs	228	Layer 1 rule-based keyword + scoring engine
6	PAM_AIEngine.gs	258	Layer 2 OpenAI gpt-4o-mini evaluation
7	PAM_Menu.gs	61	Menu registration and test data seeder

#	File	Lines	Purpose
8	PAM_UI.html	1,654	Full Command Center dashboard (5 tabs)

HTML UI Components (18 files)

File	Type	Purpose
AppointmentManager.html	Modal	Schedule test drives, deliveries, inspections
CallLogs.html	Modal	Call history with inbound/outbound/missed tracking
CampaignManager.html	Modal	SMS/Email campaign management
CRMAnalytics.html	Modal	KPIs, conversion rates, sales funnel
DealAnalyzer.html	Modal	Evaluate deals with spread/margin/risk
DealCalculator.html	Modal	Real-time profit calculator
DealGallery.html	Full Page	Card-view deal gallery with actions
FollowUpCenter.html	Modal	Active follow-up monitoring
FollowUpSequences.html	Modal	Automated sequence management
IntegrationManager.html	Modal	Third-party service connections
KnowledgeBase.html	Modal	Vehicle data and market insights
PipelineView.html	Modal	Kanban-style deal pipeline
QuantumHelp.html	Modal	Guides, tips, keyboard shortcuts
QuickActions.html	Sidebar	One-click common tasks
Settings.html	Sidebar	System preferences, API keys, thresholds
SMSConversations.html	Modal	SMS thread viewer
SpeedToLead.html	Modal	Lead response time tracking
VINDecoder.html	Modal	VIN decode with vehicle details

Config Files

File	Purpose
appsscript.json	GAS manifest: timezone, scopes, runtime
.clasp.json	clasp CLI: script ID, push order (56 files)

D) OAuth Scopes Required

Scope	Purpose
spreadsheets.currentonly	Current spreadsheet access
spreadsheets	Full Sheets access
drive	Google Drive (file exports)
script.external_request	HTTP requests (OpenAI, Browse.AI, SMS-iT, etc.)
script.scriptapp	Trigger management
userinfo.email	User identification
gmail.send	Email notifications and campaigns

Scope	Purpose
script.container.ui	Dialogs, modals, sidebars

E) Global Configuration Constants

```
// quantum_config.gs
const QUANTUM = {
VERSION: 'QUANTUM-2.0.0',
NAME: 'CarHawk Ultimate CRM',
SIGNATURE: '■■■'
}

// quantum_core.gs
const QUANTUM_CONFIG = {
HOME_COORDINATES: { lat: 38.6270, lng: -90.1994 },
HOME_ZIP: '63101',
ANALYSIS_DEPTH: 'QUANTUM', // BASIC | ADVANCED | QUANTUM
PREDICTION_WINDOW: 30, // days
MARKET_REFRESH_RATE: 3600, // seconds
AI_CONFIDENCE_THRESHOLD: 0.85, // 85%
PROFIT_QUANTUM: 2000, // min profit threshold ($)
VELOCITY_THRESHOLD: 14, // days for quick flip
REPAIR_KEYWORDS: [
{ keyword: 'transmission', severity: 'HIGH', cost: 3000 },
{ keyword: 'engine knock', severity: 'HIGH', cost: 2500 },
{ keyword: 'needs motor', severity: 'CRITICAL', cost: 4000 },
{ keyword: 'blown head', severity: 'HIGH', cost: 1500 },
{ keyword: 'no reverse', severity: 'MEDIUM', cost: 2000 },
{ keyword: 'overheating', severity: 'MEDIUM', cost: 800 },
{ keyword: 'ac broken', severity: 'LOW', cost: 500 },
{ keyword: 'minor dents', severity: 'LOW', cost: 300 }
]
}

const CAPITAL_TIERS = {
MICRO: { min: 0, max: 1000, label: 'Micro Flip', multiplier: 2.5 },
BUDGET: { min: 1000, max: 4000, label: 'Budget Flip', multiplier: 2.0 },
STANDARD: { min: 4000, max: 10000, label: 'Standard Flip', multiplier: 1.5 },
DEALER: { min: 10000, max: Infinity, label: 'Dealer Flip', multiplier: 1.2 }
}
```

F) Global State Object

```
const QuantumState = {
analysisQueue: [],
activeProcessors: 0,
maxProcessors: 5,
marketIntelligence: {},
predictiveModels: {},
realTimeAlerts: [],
campaignQueue: [],
followUpQueue: []
}
```

G) Settings Sheet Keys

Settings stored in the Settings sheet (key-value):

Key	Default	Type	Purpose
BUSINESS_NAME	(required)	String	Company name for branding
HOME_ZIP	63101	String	Base location for distance calc
OPENAI_API_KEY	(required)	String	GPT-4 API access
SMSIT_API_KEY	(optional)	String	SMS-iT API key
SMSIT_WEBHOOK_URL	(optional)	URL	SMS-iT webhook endpoint
OHMYLEAD_WEBHOOK_URL	(optional)	URL	Ohmylead sync endpoint
SENDGRID_API_KEY	(optional)	String	Email delivery
TWILIO_ACCOUNT_SID	(optional)	String	Fallback SMS
TWILIO_AUTH_TOKEN	(optional)	String	Fallback SMS
TWILIO_PHONE	(optional)	String	Fallback SMS sender
PROFIT_TARGET	2000	Number	Min profit for alerts
ANALYSIS_DEPTH	Quantum	String	AI depth level
ALERT_EMAIL	(required)	Email	Notification recipient
REALTIME_MODE	false	Boolean	Enable hourly sync
REALTIME_SYNC	false	Boolean	Sync enabled flag
SYNC_INTERVAL	300	Number	Sync interval (seconds)
REALTIME_ALERTS	true	Boolean	Enable push alerts
CRM_ENABLED	false	Boolean	Enable CRM sync
ALERTS_ENABLED	true	Boolean	Enable alert system
AUTO_FOLLOW_UP	true	Boolean	Auto-create follow-ups for hot deals
YOUR_NAME	(optional)	String	Name for email templates
SYSTEM_VERSION	1.0.0	String	Installed version
INSTALL_DATE	(auto)	ISO Date	First deployment
LAST_SYNC	(auto)	ISO Date	Last hourly sync
LAST_ANALYSIS	(auto)	ISO Date	Last daily analysis

Plus 20 Turo-specific settings (see TURO_SPEC_PACK.md).

Plus 12 PAM-specific settings (see SPEC_PACK_PART9_PAM.md).

H) Menu Structure

Main Menu: ■■ CarHawk Ultimate

Submenu	Items	Key Functions
■■ Quantum Operations	6	initializeQuantumSystem, quantumImportSync, executeQuantumAIBatch, toggleRealTimeMode, analyzeQuantumDeal, runDeepMarketScan
■ CRM Operations	7	openAppointmentManager, openFollowUpCenter, openCampaignManager, openSMSConversations, openCallLogs, launchCampaignUI, openCRMAnalytics

Submenu	Items	Key Functions
■ CRM & Export	5	exportQuantumSMS, exportQuantumCRM, generateQuantumCampaigns, syncQuantumCRM, exportQuantumAnalytics
■ Lead Management	7	openLeadTracker, openLeadScoring, viewHotLeads, viewColdLeads, openSpeedToLead, openPipelineView
■ Alerts & Automation	6	checkQuantumAlerts, sendQuantumDigest, configureQuantumTriggers, showAutomationStatus, openScheduleManager, manageFollowUpSequences
■ Analytics & Reports	7	openQuantumDashboard, generatePerformanceMatrix, generateMarketIntelligence, generateQuantumWeekly, generateQuantumMonthly, runROIOptimizer, generateClosedDealsReport
■ Browse.ai Robots	9	setBrowseAIApiKeyUI, showBrowseAIRobotsUI, linkBrowseAIRobotUI, registerRobotUI, showRobotSetupGuide, deployRobotUI, fetchAndImportBrowseAIDataUI, importFromBrowseAI, showRobotStatusUI
■■ Tools & Utilities	7	openQuantumVINDecoder, openDealCalculatorPro, generateMarketHeatMap, openKnowledgeBase, runSystemDiagnostics, openQuantumSettings, openIntegrationManager
■ Turo Module	8	analyzeTuroSelected, batchAnalyzeTuro, refreshFleetManager, addToFleetSelected, updateFleetFinancials, logMaintenanceEvent, checkComplianceAlerts, setupTuroModule

PAM Menu (separate top-level):

Item	Function
Open Command Center	openPAMDashboard
Run Logic Engine	runPAMLogicEngine
Run AI Evaluation	runPAMAIEvaluation
Full Cycle (Logic → AI)	runPAMFullCycle
Initialize PAM Sheets	initializePAMSheets

Quick Access (top-level):

- Deal Gallery, Quick Actions, Deal Analyzer, Quantum Help, About CarHawk Ultimate

---

I) System Initialization (8-Phase Deploy)

`deployQuantumArchitecture(config)` runs these phases in order:

Phase	Function	What It Does
1	createQuantumSheets()	Creates 20 sheets with colors, icons, protection
2	deployQuantumHeaders()	Deploys 444+ column headers across 18 sheets
3	deployQuantumFormulas()	Deploys ROI/scoring formulas (placeholder)
4	initializeAIModels(config)	Stores API keys and config to Settings
5	setupRealtimeSync()	Initializes sync settings
6	initializeCRMSystem(config)	Sets up CRM credentials and defaults
7	deployQuantumTriggers()	Creates hourly, daily, and CRM triggers
8	generateQuantumDashboard()	Builds initial dashboard

J) clasp Push Order

Files deploy in this dependency order (56 total):

1. quantum_config.gs (identity)
2. quantum_core.gs (constants)
3. quantum_utilities.gs (helpers)
4. quantum_headers.gs → quantum_setup.gs → quantum_menu.gs
5. All remaining quantum_*.gs modules
6. All turo_*.gs modules
7. All *.html UI files

CarHawk Ultimate — Sheet Architecture & Column Schemas

Spec Pack Part 2 of 8 | QUANTUM-2.0.0

A) Sheet Inventory (20 Core + 5 Turo)

#	Sheet Name	Columns	Tab Color	Icon	Source File
1	Master Import	19	#4285f4	■	quantum_headers.gs
2	Master Database	61	#0f9d58	■■	quantum_headers.gs
3	Verdict	24	#ea4335	■■	quantum_headers.gs
4	Leads Tracker	29	#fbbc04	■	quantum_headers.gs
5	Flip ROI Calculator	32	#673ab7	■	quantum_headers.gs
6	Lead Scoring & Risk Assessment	26	#ff6d00	■	quantum_headers.gs
7	CRM Integration	24	#00acc1	■	quantum_headers.gs
8	Parts Needed	31	#795548	■	quantum_headers.gs
9	Post-Sale Tracker	34	#607d8b	■	quantum_headers.gs
10	Reporting & Charts	12	#9e9e9e	■	quantum_headers.gs
11	Settings	11	#424242	■■	quantum_headers.gs
12	Activity Logs	11	#212121	■	quantum_headers.gs
13	Appointments	20	#4caf50	■	quantum_headers.gs
14	Follow Ups	20	#ff9800	■	quantum_headers.gs
15	Campaign Queue	20	#9c27b0	■	quantum_headers.gs
16	SMS Conversations	20	#00bcd4	■	quantum_headers.gs
17	AI Call Logs	20	#f44336	■	quantum_headers.gs
18	Closed Deals	28	#4caf50	■	quantum_headers.gs
19	Knowledge Base	20	#3f51b5	■	quantum_headers.gs
20	Integrations	19	#009688	■	quantum_headers.gs

#	Sheet Name	Columns	Tab Color	Icon	Source File
21	Turo Engine	39	#00BCD4	--	turo_setup.gs
22	Fleet Manager	26	#4CAF50	--	turo_setup.gs
23	Maintenance & Turnovers	14	#FF9800	--	turo_setup.gs
24	Turo Pricing & Seasonality	Matrix	#9C27B0	--	turo_setup.gs
25	Insurance & Compliance	17	#F44336	--	turo_setup.gs

Notes:

- Sheets 21-25 are Turo Module sheets -- see TURO_SPEC_PACK.md for full schemas.
- System sheets (Settings, Activity Logs, Integrations) have warning-only protection.
- All headers are frozen in row 1, bold, white text, colored backgrounds, Google Sans 11pt.

B) Master Import (19 columns)

Source: `setupImportHeaders()` in quantum_headers.gs

Col	Header	Type	Source
A (0)	Import ID	String	Auto-generated
B (1)	Date (GMT)	DateTime	Import timestamp
C (2)	Job Link	URL	Browse.AI job URL
D (3)	Origin URL	URL	Original listing URL
E (4)	Platform	String	Detected from URL
F (5)	Raw Title	String	Listing title as scraped
G (6)	Raw Price	String	Price text as scraped
H (7)	Raw Location	String	Location as scraped
I (8)	Raw Description	String	Enhanced with [TAG: value]
J (9)	Seller Info	String	Enhanced with type detection
K (10)	Posted Date	String	Original post date
L (11)	Images Count	Number	Photo count
M (12)	Raw Mileage	String	Mileage as scraped
N (13)	Raw Year	String	Year as scraped
O (14)	Raw Condition	String	Condition as scraped
P (15)	Import Status	String	Processing status
Q (16)	Processed	Boolean	Has been synced to DB
R (17)	Master ID	String	Linked Deal ID in DB
S (18)	Error Log	String	Any import errors

C) Master Database (61 columns)

Source: `setupDatabaseHeaders()` in quantum_headers.gs

This is the central deal repository. Every deal lives here after import processing.

Col	Header	Type	Category
A (0)	Deal ID	String	Metadata
B (1)	Import Date	DateTime	Metadata
C (2)	Platform	String	Metadata
D (3)	Status	String	Metadata
E (4)	Priority	String	Metadata
F (5)	Year	Number	Vehicle Specs
G (6)	Make	String	Vehicle Specs
H (7)	Model	String	Vehicle Specs
I (8)	Trim	String	Vehicle Specs
J (9)	VIN	String	Vehicle Specs
K (10)	Mileage	Number	Vehicle Specs
L (11)	Color	String	Vehicle Specs
M (12)	Title	String	Listing Info
N (13)	Price	Currency	Listing Info
O (14)	Location	String	Listing Info
P (15)	ZIP	String	Listing Info
Q (16)	Distance	Number	Listing Info
R (17)	Location Risk	String	Listing Info
S (18)	--	--	--
T (19)	Condition	String	Condition
U (20)	Condition Score	Number	Condition
V (21)	Repair Keywords	String	Condition
W (22)	Repair Risk Score	Number	Condition
X (23)	Est. Repair Cost	Currency	Condition
Y (24)	--	--	--
Z (25)	Market Value	Currency	Market Analysis
AA (26)	MAO	Currency	Market Analysis
AB (27)	Profit Margin	Currency	Market Analysis
AC (28)	ROI %	Percent	Market Analysis
AD (29)	Capital Tier	String	Market Analysis
AE (30)	Flip Strategy	String	Sales Velocity
AF (31)	Sales Velocity Score	Number	Sales Velocity
AG (32)	Market Advantage	Number	Sales Velocity
AH (33)	Days Listed	Number	Sales Velocity
AI (34)	Seller Name	String	Seller Info
AJ (35)	Seller Phone	String	Seller Info
AK (36)	Seller Email	String	Seller Info
AL (37)	Seller Type	String	Seller Info
AM (38)	Deal Flag	String	Deal Flags
AN (39)	Hot Seller?	Boolean	Deal Flags
AO (40)	Multiple Vehicles?	Boolean	Deal Flags

Col	Header	Type	Category
AP (41)	Seller Message	String	Deal Flags
AQ (42)	AI Confidence	Number	Verdict
AR (43)	Verdict	String	Verdict
AS (44)	Verdict Icon	String	Verdict
AT (45)	Recommended?	String	Verdict
AU (46)	Image Score	Number	Engagement
AV (47)	Engagement Score	Number	Engagement
AW (48)	Competition Level	Number	Engagement
AX (49)	Last Updated	DateTime	CRM Fields
AY (50)	Assigned To	String	CRM Fields
AZ (51)	Notes	String	CRM Fields
BA (52)	Stage	String	CRM Fields
BB (53)	Contact Count	Number	CRM Fields
BC (54)	Last Contact	DateTime	CRM Fields
BD (55)	Next Action	String	CRM Fields
BE (56)	Response Rate	Percent	CRM Fields
BF (57)	SMS Count	Number	CRM Fields
BG (58)	Call Count	Number	CRM Fields
BH (59)	Email Count	Number	CRM Fields
BI (60)	Meeting Scheduled	Boolean	CRM Fields
BJ (61)	Follow-up Status	String	CRM Fields

Turo Writeback Columns (appended by Turo Module -- BJ-BS):

Col	Header	Type
BK	Turo Hold Score	Integer 0-100
BL	Turo Monthly Net	Currency
BM	Turo Payback Months	Decimal
BN	Turo Break-Even Util %	Percent
BO	Turo Risk Tier	String
BP	Turo vs Flip Delta	Currency
BQ	Turo Recommended?	String
BR	Turo Status	String
BS	Fleet ID	String
BT	Turo Notes	String

Deal Stages (pipeline lifecycle):

```
IMPORTED → CONTACTED → RESPONDED → SCHEDULING → APPOINTMENT_SET → NEGOTIATING → CLOSED_WON
→ LOST
```

Verdict Values:

- ■ HOT DEAL
- ■ SOLID DEAL
- ■■ PORTFOLIO FOUNDATION
- ■ PASS

D) Verdict (24 columns)

Source: `setupVerdictHeaders()` in quantum_headers.gs

Col	Header	Type
A	Analysis ID	String
B	Deal ID	String
C	Analysis Date	DateTime
D	Model Version	String
E	Quantum Score	Number 0-100
F	AI Verdict	String
G	Confidence %	Number
H	Profit Potential	Currency
I	Risk Assessment	String
J	Market Timing	String
K	Competition Analysis	String
L	Price Optimization	String
M	Negotiation Strategy	String
N	Quick Sale Probability	Percent
O	Repair Complexity	String
P	Hidden Cost Risk	Number 0-100
Q	Flip Timeline	String
R	Success Probability	Percent
S	Alternative Strategies	String
T	Key Insights	String
U	Red Flags	String
V	Green Lights	String
W	Market Comparables	String
X	Decision Matrix	String

E) Leads Tracker (29 columns)

Source: `setupLeadsTrackerHeaders()` in quantum_headers.gs

Col	Header	Type
A	Lead ID	String
B	Deal ID	String
C	Date Added	DateTime
D	Status	String
E	Priority	String (High/Medium/Low)
F	Stage	String
G	Vehicle	String

Col	Header	Type
H	Price	Currency
I	Profit Potential	Currency
J	ROI %	Percent
K	Location	String
L	Distance	Number
M	Seller Name	String
N	Phone	String
O	Email	String
P	Best Contact Time	String
Q	Contact Attempts	Number
R	Last Contact	DateTime
S	Next Action	String
T	Action Date	DateTime
U	Response Rate	Percent
V	Interest Level	String
W	Negotiation Notes	String
X	Final Offer	Currency
Y	Close Probability	Percent
Z	Assigned To	String
AA	Tags	String
AB	Follow-up Required	Boolean
AC	SMS/Email Sent	Boolean

F) Flip ROI Calculator (32 columns)

Source: `setupROICalculatorHeaders()` in quantum_headers.gs

Col	Header	Type
A	Calc ID	String
B	Deal ID	String
C	Vehicle	String
D	Scenario	String
E	Purchase Price	Currency
F	Transport Cost	Currency
G	Inspection Cost	Currency
H	Repair Labor	Currency
I	Parts Cost	Currency
J	Detail Cost	Currency
K	Marketing Cost	Currency
L	Listing Fees	Currency
M	Other Costs	Currency

Col	Header	Type
N	Total Investment	Currency
O	Target Sale Price	Currency
P	Market Comp Avg	Currency
Q	Days to Sell Est	Number
R	Holding Cost/Day	Currency
S	Total Holding	Currency
T	Transaction Fees	Currency
U	Total Costs	Currency
V	Net Profit	Currency
W	Profit Margin %	Percent
X	ROI %	Percent
Y	Cash on Cash	Percent
Z	Break Even Price	Currency
AA	Min Acceptable	Currency
AB	Max Acceptable	Currency
AC	Risk Score	Number
AD	Confidence Level	Percent
AE	Scenario Notes	String

G) Lead Scoring & Risk Assessment (26 columns)

Source: `setupScoringHeaders()` in quantum_headers.gs

Col	Header	Type
A	Score ID	String
B	Deal ID	String
C	Analysis Date	DateTime
D	Quantum Score	Number
E	Component Scores	String (JSON)
F	Market Score	Number
G	Vehicle Score	Number
H	Seller Score	Number
I	Timing Score	Number
J	Location Score	Number
K	Competition Score	Number
L	Profit Score	Number
M	Risk Score	Number
N	Velocity Score	Number
O	Condition Score	Number
P	Total Weighted Score	Number
Q	Percentile Rank	Number

Col	Header	Type
R	Category	String
S	Investment Grade	String
T	Risk Factors	String
U	Opportunity Factors	String
V	Score Trend	String
W	Previous Score	Number
X	Score Change	Number
Y	Analyst Notes	String
Z	Override / Final Grade	String

H) CRM Integration (24 columns)

Source: `setupCRMHeaders()` in quantum_headers.gs

Col	Header	Type
A	Export ID	String
B	Export Date	DateTime
C	Platform	String
D	Deal IDs	String (CSV)
E	Record Count	Number
F	Export Type	String
G	Include Fields	String
H	Filters Applied	String
I	Total Value	Currency
J	Avg Deal Value	Currency
K	Hot Leads Count	Number
L	Contact Info Complete	Boolean
M	SMS/Email Ready	Boolean
N	Campaign Name	String
O	Template Used	String
P	Tags Applied	String
Q	CRM Record IDs	String
R	Sync Status	String
S	Sync Errors	String
T	Last Sync	DateTime
U	Next Sync	DateTime
V	Automation Enabled	Boolean
W	Response Tracking	Boolean
X	Conversion Count / Revenue	Number

I) Parts Needed (31 columns)

Source: `setupPartsHeaders()` in quantum_headers.gs

Col	Header	Type
A	Part ID	String
B	Deal ID	String
C	Vehicle	String
D	Category	String
E	Subcategory	String
F	Part Name	String
G	Part Number	String
H	OEM Number	String
I	Condition Needed	String
J	Price (New)	Currency
K	Price (Used)	Currency
L	Price (Reman)	Currency
M	Selected Option	String
N	Quantity	Number
O	Unit Cost	Currency
P	Total Cost	Currency
Q	Supplier Name	String
R	Supplier Contact	String
S	Lead Time (days)	Number
T	Stock Status	String
U	Order Date	DateTime
V	Delivery Date	DateTime
W	Quality Check	Boolean
X	Labor Hours	Number
Y	Labor Rate	Currency
Z	Labor Total	Currency
AA	Core Charge	Currency
AB	Warranty	String
AC	Notes	String

J) Post-Sale Tracker (34 columns)

Source: `setupPostSaleHeaders()` in quantum_headers.gs

Col	Header	Type
A	Sale ID	String
B	Deal ID	String
C	Vehicle	String

Col	Header	Type
D	Sale Date	DateTime
E	Days to Sell	Number
F	Buyer Name	String
G	Buyer Type	String
H	Sale Platform	String
I	Listed Price	Currency
J	Sale Price	Currency
K	Negotiation %	Percent
L	Purchase Price	Currency
M	Total Investment	Currency
N	Gross Profit	Currency
O	Net Profit	Currency
P	Actual ROI	Percent
Q	Projected ROI	Percent
R	Payment Method	String
S	Payment Status	String
T	Title Transfer	Boolean
U	Delivery Method	String
V	Buyer Satisfaction	Number 1-5
W	Lessons Learned	String
X	Strategy Used	String
Y	Market Conditions	String
Z	Seasonal Impact	String
AA	Repeat Buyer	Boolean
AB	Referral Source	String
AC	Follow-up Date	DateTime
AD	Testimonial	String
AE	Case Study	Boolean
AF	Performance Grade	String

K) Remaining Sheets (Compact)

Reporting & Charts (12 columns)

Metric, Value, Change, Trend, Target, Status, Period, Comparison, Percentile, Grade, Action Required, Notes

Settings (11 columns)

Setting Key, Value, Updated, Description, Category, Data Type, Validation, Default, Required, Affects, Restart Required

Activity Logs (11 columns)

Timestamp, Level, Action, Category, Details, User, Deal ID, Duration (ms), Success, Error, Stack Trace

Appointments (20 columns)

Appointment ID, Deal ID, Vehicle, Seller Name, Phone, Email, Scheduled Time, Location, Location Type, Duration, Status, Type, Notes, Reminder Sent, Created Date, Created By, Updated Date, Confirmed, Show Rate, Outcome, Follow-up Required

Follow Ups (20 columns)

Follow-up ID, Deal ID, Campaign ID, Sequence Type, Step Number, Scheduled Time, Type, Template, Status, Sent Time, Response, Response Time, Opened, Clicked, Replied, Created Date, Priority, Retry Count, Error Message, Next Step

Campaign Queue (20 columns)

Touch ID, Campaign ID, Deal ID, Sequence Type, Touch Index, Type, Template, Subject, Message, Scheduled Time, Status, Sent Time, Delivered, Response, Response Type, Created Date, Tags, A/B Test, Performance Score, Cost

SMS Conversations (20 columns)

Conversation ID, Deal ID, Phone Number, Direction, Message, Timestamp, Status, Intent, Sentiment, Type, Campaign ID, Template Used, Response Time, Character Count, Media URL, Error Code, Cost, Provider, Thread ID, Tags

AI Call Logs (20 columns)

Call ID, Deal ID, Phone Number, Direction, Start Time, End Time, Duration, Recording URL, Transcription, Summary, Sentiment, Intent, Outcome, Next Action, Appointment Detected, Price Discussed, Objections, AI Score, Cost, Tags

Closed Deals (28 columns)

Close ID, Deal ID, Vehicle, Year, Make, Model, Mileage, Condition, Original Price, Purchase Price, Sale Price, Platform, Days to Close, Days on Market, Profit, ROI %, Close Date, Payment Method, Buyer Type, Marketing Cost, Repair Cost, Total Investment, Net Profit, Commission, Success Factors, Lessons Learned, Rating, Tags

Knowledge Base (20 columns)

KB ID, Make, Model, Years, Category, Common Issues, Repair Costs, Market Demand, Quick Flip Score, Avg Days to Sell, Price Range Low, Price Range High, Best Months, Target Buyer, Negotiation Tips, Red Flags, Success Rate, Updated Date, Data Points, Confidence Score

Integrations (19 columns)

Integration ID, Provider, Type, Name, API Key, Secret, Status, Last Sync, Next Sync, Sync Frequency, Records Synced, Error Count, Last Error, Configuration, Webhook URL, Features, Limits, Cost, Notes, Created Date

---

L) Header Formatting Standard

Applied by `applyQuantumHeaders()` to every sheet:

Property	Value
Font	Google Sans, 11pt
Weight	Bold
Text Color	White
Background	Sheet-specific color
Alignment	Center, Wrap
Row Height	40px

Property	Value
Frozen Rows	1 (header row)
Auto-resize	All columns

CarHawk Ultimate — AI Engine & Deal Analysis

Spec Pack Part 3 of 8 | QUANTUM-2.0.0

A) AI Analysis Pipeline

Source Files: quantum_ai.gs (299 lines), quantum_calculations.gs (371 lines), quantum_knowledge_base.gs (65 lines), quantum_fallback.gs (92 lines)

Flow:

```
Master Database row
↓
calculateQuantumMetrics(parsed) → ROI, MAO, scores, distance, condition
↓
prepareQuantumContext(dealData, metrics) → structured context object
↓
executeQuantumAnalysis(context, apiKey, depth) → OpenAI GPT-4 call
↓
updateQuantumResults(sheet, rowNum, analysis) → write verdicts to DB
↓
logQuantumVerdict(verdictSheet, dealId, analysis) → write to Verdict sheet
↓
checkQuantumAlerts(dealData, analysis) → trigger alerts if hot
↓
createFollowUpSequence(dealId, 'HOT_LEAD') → auto-engage if enabled
```

B) OpenAI Integration

File: quantum_ai.gs

API Configuration

Setting	Value
Endpoint	`https://api.openai.com/v1/chat/completions`
Model (QUANTUM)	gpt-4-turbo-preview
Model (ADVANCED)	gpt-4
Temperature	0.3 (deterministic)
Max Tokens	1000
Response Format	JSON object

System Prompt

> "You are a quantum-class vehicle investment analyst with deep market knowledge and predictive capabilities. Analyze deals with extreme precision and provide actionable intelligence."

Analysis Depth Levels

Level	Prompt Enhancement
QUANTUM	"Ultra-deep analysis considering market microtrends, seasonal patterns, demographic targeting, psychological pricing"
ADVANCED	"Profit optimization, risk mitigation, market timing focus"
BASIC	"Quick assessment of obvious profit potential and major risks"

Context Object Sent to AI

```
{
vehicle: {
year, make, model, trim, mileage, color, title
},
pricing: {
askingPrice, marketValue, mao, estimatedRepairCost
},
condition: {
stated, // e.g. "Good"
score, // 0-100
repairKeywords, // matched repair items
repairRisk // severity score
},
market: {
daysListed, platform, location, distance,
competitionLevel, salesVelocity, marketAdvantage
},
seller: {
type, // Private | Dealer
hotSeller, // boolean
multipleVehicles, // boolean
engagementScore // 0-100
}
}
```

AI Response JSON Schema

```
{
quantumScore: 0-100,
verdict: "■ HOT DEAL | ■ SOLID DEAL | ■■ PORTFOLIO FOUNDATION | ■ PASS",
confidence: 0-100,
flipStrategy: "Quick Flip | Repair + Resell | Wholesale | Part Out | Pass",
profitPotential: number,
riskAssessment: {
overall: "Low | Medium | High",
factors: ["string"]
},
marketTiming: "Excellent | Good | Fair | Poor",
priceOptimization: {
suggestedOffer: number,
maxOffer: number,
negotiationRoom: number (%)
},
quickSaleProbability: 0-100,
repairComplexity: "None | Simple | Moderate | Complex",
hiddenCostRisk: 0-100,
flipTimeline: "X-Y days",
successProbability: 0-100,
keyInsights: ["string"],
redFlags: ["string"],
greenLights: ["string"],
sellerMessage: "string",
recommended: boolean
}
```

Verdict Writeback to Master Database

DB Column	AI Field	Notes
29 (Flip Strategy)	flipStrategy	Quick Flip, Repair + Resell, etc.
37 (Deal Flag)	verdict emoji	■, ■, ■■, ■
40 (Seller Message)	sellerMessage	Auto-generated outreach text
41 (AI Confidence)	confidence	0-100

DB Column	AI Field	Notes
42 (Verdict)	verdict	Full verdict string
43 (Verdict Icon)	verdict icon	Emoji only
44 (Recommended)	recommended	YES / NO

Verdict Row Colors

Verdict	Background Color
■ HOT DEAL	#ffebee (light red)
■ SOLID DEAL	#e8f5e9 (light green)
■ PASS	#f5f5f5 (light grey)

Auto-Follow-Up Trigger

When `verdict = "■ HOT DEAL"` AND `AUTO_FOLLOW_UP = "true"`:

- Automatically calls `createFollowUpSequence(dealId, 'HOT_LEAD')`

C) Batch Analysis

Function: `executeQuantumAIBatch()`

- Reads all rows in Master Database
- Filters to unanalyzed deals (verdict column empty)
- Processes each deal through full pipeline
- Shows processing dialog with progress
- Returns `{ success, message, analyzed, duration }`
- Logs completion to Activity Logs

D) Fallback Analysis

File: quantum_fallback.gs

When OpenAI fails (network error, quota exceeded, etc.), `generateFallbackAnalysis()` returns safe defaults:

```
{
score: 50, // Neutral
verdict: 'PORTFOLIO FOUNDATION',
confidence: 0, // Signals AI failure
flipStrategy: 'Needs Review', // Escalation flag
sellerMessage: 'Thank you for your listing. We are currently reviewing
this vehicle and will follow up with more details shortly.',
context: context || {} // Preserves original for debugging
}
```

E) Calculation Engine

File: quantum_calculations.gs (371 lines)

Master Function: `calculateQuantumMetrics(parsed)`

Returns a metrics object with all computed values for a deal.

Distance Calculation

```
// calculateQuantumDistance(targetZip)
distance = Math.abs(homeZip - targetZip) * 0.1 // approximate miles
```

Location Risk Assessment

Distance	Risk	Flag	Score
< 25 mi	Low	■	90
25-75 mi	Moderate	■	60
> 75 mi	High	■	30

Condition Scoring

Condition	Score
Excellent	95
Very Good	85
Good	75
Fair	60
Poor	40
Unknown	50

Repair Risk Analysis

`analyzeRepairRisk(repairKeywords)` scans description for keywords:

Keyword	Severity	Est. Cost
transmission	HIGH	$3,000
engine knock	HIGH	$2,500
needs motor	CRITICAL	$4,000
blown head	HIGH	$1,500
no reverse	MEDIUM	$2,000
overheating	MEDIUM	$800
ac broken	LOW	$500
minor dents	LOW	$300

Returns `{ score, totalCost }` -- score is sum of severity weights.

Market Value Estimation

`estimateQuantumMarketValue(parsed)` -- 4-layer calculation:

Layer 1 -- Base Value by Age:

Vehicle Age	Base Value
0-2 years	$40,000
2-4 years	$30,000
4-6 years	$22,000
6-10 years	$15,000
10-15 years	$8,000
15+ years	$4,000

Layer 2 -- Make Premium Multipliers:

Make	Multiplier		Make	Multiplier
Porsche	1.50x		Toyota	1.10x
Tesla	1.40x		Honda	1.08x
Lexus	1.30x		Ford	0.95x
Mercedes	1.28x		Chevrolet	0.93x
BMW	1.25x		Nissan	0.92x
Audi	1.22x			

Layer 3 -- Mileage Adjustment:

Expected mileage = age x 12,000 miles/year

Deviation	Multiplier
> 20K over expected	0.85x
10-20K over	0.92x
10-20K under	1.08x
> 20K under	1.15x

Layer 4 -- Condition Multiplier:

Condition	Multiplier
Excellent	1.15x
Very Good	1.08x
Good	1.00x
Fair	0.85x
Poor	0.65x

Knowledge Base Adjustment:

- Very High demand: +10%
- High demand: +5%
- Low demand: -5%
- Capped within knowledge base price range

MAO (Maximum Allowable Offer)

```
MAO = (Market Value × 0.75) - Repair Costs - $500 Holding - (Market Value × 0.15 Profit)
Minimum: $500
```

Capital Tier Classification

Tier	Price Range	Multiplier
MICRO	$0 - $1,000	2.5x
BUDGET	$1,000 - $4,000	2.0x
STANDARD	$4,000 - $10,000	1.5x
DEALER	$10,000+	1.2x

Sales Velocity Scoring (0-100)

Base scores for popular models:

- F-150: 92, Camry: 90, Accord: 88, RAV4: 87
- Civic: 85, CR-V: 84, Silverado: 80, Wrangler: 78

Adjustments:

- +10 for Excellent/Very Good condition
- -15 if listed > 30 days
- +10 if listed < 7 days
- Knowledge base override if available

Market Advantage Score (0-100)

Base: 50 points

Factor	Adjustment
Price < 70% of market	+20
Price < 80% of market	+10
Price > 95% of market	-20
Distance < 25 miles	+10
Distance > 100 miles	-15
Condition score > 80	+15
Condition score < 50	-15
Facebook	+5
OfferUp	+3
Craigslist	0
eBay	-5

Image Scoring

Image Count	Score
15+	95
10-14	85
7-9	75
5-6	65
3-4	50
1-2	30
0	10

Engagement Score (0-100)

Base: 50

Factor	Adjustment
Listed < 3 days	+20
Listed < 7 days	+10
Listed > 30 days	-20
Private seller	+10
Has phone	+15
Has email	+10

Factor	Adjustment
Multiple vehicles	-15

Competition Level by Platform

Platform	Base Score
eBay	80 (national)
Facebook	60 (regional)
OfferUp	50 (local/regional)
Craigslist	40 (local)

- Popular models: +15
- Price < $5,000: +10
- Price > $20,000: -10

Profit Calculation

```
Profit = Market Value - Asking Price - Repair Cost - $500 Holding
Profit Margin % = (Profit / Market Value) × 100
ROI % = (Profit / (Asking Price + Repair Cost + $500)) × 100
```

Priority Classification

```
Score = (ROI × 0.30) + (Profit Margin × 0.20) + (Sales Velocity × 0.20)
+ (Market Advantage × 0.15) + ((100 - Repair Risk) × 0.15)

High: > 70
Medium: 40-70
Low: < 40
```

F) Knowledge Base

File: quantum_knowledge_base.gs (65 lines)

Pre-loaded vehicle profiles for data-driven analysis:

Make	Model	Years	Common Issues	Flip Score	Avg Days	Demand
Honda	Civic	2016-2020	AC Compressor ($1,200)	85	12	Very High
Toyota	Camry	2015-2020	None common	90	10	Very High
Honda	Accord	2016-2020	Turbo issues ($1,500)	88	14	High
Ford	F-150	2015-2020	Cam phasers ($2,000)	82	18	Very High
Chevrolet	Silverado	2014-2019	AFM issues	80	20	High
Toyota	RAV4	2016-2020	None common	88	11	Very High
Honda	CR-V	2016-2020	Oil dilution ($750)	86	13	High
Nissan	Altima	2015-2020	CVT failure ($4,000)	65	25	Medium
Mazda	CX-5	2016-2020	None common	84	15	High
Jeep	Wrangler	2015-2020	Death wobble	78	22	High

Each entry also includes: price range (low/high), best selling months, target buyer, negotiation tips, red flags, and success rate.

G) Hot Seller Detection

File: quantum_utilities.gs

`detectHotSeller(data)` flags sellers with urgency signals:

Keywords: must go, moving, asap, today, quick sale, obo, need gone

Returns `true` if posted within 24 hours OR contains urgency keyword.

`detectMultipleVehicles(description)` flags dealers:

Keywords: other vehicles, also have, check my other, more cars, inventory, dealer, lot

H) Platform Detection

`detectPlatform(url)` identifies source from URL:

URL Contains	Platform
facebook.com	Facebook
craigslist.org	Craigslist
offerup.com	OfferUp
ebay.com	eBay
(other)	Unknown

Advanced detection via `detectPlatformFromURLAdvanced()` supports 10+ platforms using ROBOT_REGISTRY URL patterns.

I) Make Standardization

`standardizeMake(make)` normalizes abbreviations:

Input	Output
Chevy	Chevrolet
VW	Volkswagen

CarHawk Ultimate — CRM Engine & Automation

Spec Pack Part 4 of 8 | QUANTUM-2.0.0

A) CRM Architecture Overview

Source Files: quantum_crm_engine.gs (379 lines), quantum_crm_helpers.gs (220 lines), quantum_crm_automation.gs (143 lines), quantum_crm_api.gs (139 lines)

Pipeline Stages:

```
IMPORTED → CONTACTED → RESPONDED → SCHEDULING → APPOINTMENT_SET → NEGOTIATING → CLOSED_WON
→ LOST
```

CRM Capabilities:

- Appointment scheduling with reminders
- Automated follow-up sequences (SMS + Email)
- SMS conversation logging
- AI call logging with transcription analysis
- Multi-deal campaign management
- Closed deal recording with profit tracking
- Lead scoring and pipeline management
- Multi-channel outreach (SMS, Email, Phone)

B) Follow-Up Sequences

Defined in: quantum_core.gs (CRM_CONFIG)

HOT_LEAD Sequence (4 touches)

Step	Delay	Channel	Template
1	Immediate	SMS	initial_hot
2	+30 min	SMS	follow_up_1
3	+24 hours	SMS	follow_up_2
4	+72 hours	EMAIL	follow_up_3

WARM_LEAD Sequence (3 touches)

Step	Delay	Channel	Template
1	Immediate	SMS	initial_warm
2	+24 hours	SMS	follow_up_1
3	+5 days	EMAIL	follow_up_2

COLD_LEAD Sequence (2 touches)

Step	Delay	Channel	Template
1	Immediate	EMAIL	initial_cold
2	+7 days	SMS	reengagement

C) SMS Templates

Variables: `{name}`, `{year}`, `{make}`, `{model}`, `{price}`, `{vehicle}`

Template	Message
initial_hot	"Hi {name}! I saw your {year} {make} {model} for ${price}. I'm a cash buyer ready to meet today. Is it still available?"
initial_warm	"Hi {name}, interested in your {make} {model}. Is it still for sale? I can come take a look this week."
initial_cold	"Hi, is your {year} {make} {model} still available? I'm looking for something like this."
follow_up_1	"Hi {name}, following up on your {make} {model}. I'm still interested if it's available."
follow_up_2	"Last check - is your {vehicle} still for sale? I have cash ready."
reengagement	"Hi {name}, still have your {vehicle}? Market conditions have improved, I can make a better offer now."

D) CRM Engine Functions

File: quantum_crm_engine.gs (379 lines)

Function	Purpose	Returns
`initializeCRMSystem(config)`	Sets up CRM with API credentials	void
`scheduleAppointment(dealId, data)`	Creates appointment in sheet	appointmentId
`createFollowUpSequence(dealId, type)`	Creates automated follow-up	campaignId
`logSMSConversation(dealId, phone, msg, dir, intent)`	Records SMS interaction	conversationId
`logAICall(dealId, callData)`	Logs call with transcription insights	callId
`launchCampaign(dealIds, type, name)`	Launches multi-deal campaign	campaignId
`recordClosedDeal(dealId, saleData)`	Records completed sale	closeId
`calculateDaysToClose(importDate)`	Days between import and close	number
`addToLeadsTracker(dealId, parsed, metrics)`	Adds deal to lead pipeline	leadId
`generateLeadTags(parsed, metrics)`	Creates searchable tags	string (CSV)

E) CRM Helpers -- NLP & Contact Management

File: quantum_crm_helpers.gs (220 lines)

Message Intent Detection

`analyzeMessageIntent(message)` classifies seller responses:

Intent	Keywords
SOLD	"sold", "no longer available"
AVAILABLE	"yes", "still available"
SCHEDULING	"when", "time", "meet"

Intent	Keywords
NEGOTIATION	"price", "negotiable", "offer"
OPT_OUT	"stop", "remove", "unsubscribe"
GENERAL	(default)

Sentiment Analysis

`analyzeSentiment(text)` scores tone:

Category	Keywords
POSITIVE	great, excellent, perfect, yes, interested, available, sure
NEGATIVE	sold, no, not, stop, dont, remove, spam

Score > 0 = POSITIVE, < 0 = NEGATIVE, 0 = NEUTRAL

Call Analysis Functions

Function	Purpose
`generateCallSummary(transcription)`	Extracts key sentences
`analyzeCallIntent(transcription)`	APPOINTMENT_REQUEST, PRICE_INQUIRY, CONDITION_INQUIRY, GENERAL_INQUIRY
`detectAppointmentInCall(transcription)`	Boolean: appointment scheduling detected
`detectPriceDiscussion(transcription)`	`{ discussed, keywords[], pricesMentioned }`
`extractObjections(transcription)`	PRICE, CONSIDERATION, DECISION_MAKER, NO_NEED, LOCATION

Objection Patterns

Objection	Regex Pattern		
PRICE	"too high\	too much\	expensive"
CONSIDERATION	"need to think\	think about it"	
DECISION_MAKER	"check with\	ask my"	
NO_NEED	"already have\	don't need"	
LOCATION	"too far\	distance"	

AI Call Scoring (0-100)

Base: 50 points

Factor	Adjustment
Outcome = "Appointment Set"	+30
Sentiment = POSITIVE	+20
Duration > 180 seconds	+10
Sentiment = NEGATIVE	-20
Outcome = "Not Interested"	-30
Duration < 30 seconds	-10

Contact Management

Function	Purpose	DB Columns Updated
`incrementContactCount(dealId, type)`	Tracks contact metrics	52 (count), 53 (last contact), 56-58 (SMS/Call/Email counts)
`updateDealStage(dealId, newStage)`	Changes pipeline stage	51 (Stage)
`updateDealFollowUpStatus(dealId, status)`	Updates follow-up state	60 (Follow-up Status)
`getDealById(dealId)`	Retrieves deal data	Returns object with all key fields
`fillTemplate(template, deal)`	Interpolates deal data into message templates	--

Deal Object (from getDealById)

```
{
dealId, year, make, model, price,
sellerName, sellerPhone, sellerEmail,
stage, rowNum
}
```

F) CRM Automation

File: quantum_crm_automation.gs (143 lines)

Automated Triggers

Trigger	Frequency	Function
Follow-Up Processor	Every 5 minutes	`processFollowUps()`
Campaign Processor	Every 10 minutes	`processCampaigns()`
Appointment Reminders	Every hour	`processAppointmentReminders()`

Follow-Up Processing Logic

1. Query Follow Ups sheet for rows where Status = "Scheduled" AND scheduledTime <= now
2. Send via SMS or Email depending on type column
3. Update Status to "Sent" with sent timestamp
4. On error: increment retry count (col 18), log error message (col 19)

Campaign Processing Logic

1. Query Campaign Queue for rows where Status = "Scheduled" AND scheduledTime <= now
2. Look up deal's seller phone/email
3. Send SMS to sellerPhone or EMAIL to sellerEmail
4. Update Status (col 11), sent time (col 12), delivered flag (col 13)

Appointment Reminders

1. Query Appointments sheet for Status = "Scheduled" AND scheduledTime within next hour
2. Check Reminder Sent flag (col 14) is false
3. Send SMS: "Reminder: We're scheduled to meet about your {make} {model} at {time}"
4. Mark Reminder Sent = true
5. Log as REMINDER intent

G) CRM API Layer

File: quantum_crm_api.gs (139 lines)

Delivery Channels

Service	Purpose	Auth Keys	Priority
SMS-iT	Primary SMS	SMSIT_API_KEY, SMSIT_WEBHOOK_URL	1st
Twilio	Fallback SMS	TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE	2nd
SendGrid	Primary Email	SENDGRID_API_KEY	1st
Google MailApp	Fallback Email	(built-in)	2nd

Fallback Strategy

SMS delivery order:

1. SMS-iT webhook → 2. SMS-iT API → 3. Twilio

Email delivery order:

1. SendGrid → 2. Google MailApp

SMS-iT Webhook Payload

```
POST {SMSIT_WEBHOOK_URL}
{
phone: "+1XXXXXXXXXX",
message: "string",
dealId: "string",
source: "CarHawk Ultimate"
}
```

Twilio SMS Payload

```
POST https://api.twilio.com/2010-04-01/Accounts/{SID}/Messages.json
Authorization: Basic (SID:TOKEN)
{
To: phoneNumber,
From: twilioPhone,
Body: message
}
```

SendGrid Email Payload

```
POST https://api.sendgrid.com/v3/mail/send
Authorization: Bearer {SENDGRID_API_KEY}
{
personalizations: [{ to: [{ email: to }] }],
from: { email: alertEmail, name: businessName },
subject: subject,
content: [{ type: "text/html", value: body }]
}
```

H) Inbound Response Handling

File: quantum_utilities.gs

`analyzeInboundResponse(dealId, message)` processes seller replies:

1. Analyzes message intent via `analyzeMessageIntent()`
2. Analyzes sentiment via `analyzeSentiment()`
3. Updates deal stage based on intent:

- SOLD → LOST
- AVAILABLE → RESPONDED
- SCHEDULING → SCHEDULING

4. Sets response rate to 100% (column 55)
5. Logs SMS conversation

`processCallInsights(dealId, callData)` analyzes call transcriptions:

1. Detects appointment mentions → updates stage to APPOINTMENT_SET
2. Detects price discussions → logs to deal notes (column 50)

I) Email Templates

File: quantum_utilities.gs

`getEmailTemplate(templateName, deal)` returns HTML emails with variable interpolation:

Variables: `{deal.sellerName}`, `{deal.year}`, `{deal.make}`, `{deal.model}`, `{YOUR_NAME}`

Supported templates: follow_up_3, follow_up_2, initial_cold

J) Closed Deal Recording

`recordClosedDeal(dealId, saleData)` writes to Closed Deals sheet (28 columns):

- Generates Close ID (CLOSE-timestamp-random)
- Calculates days to close from import date
- Calculates profit, ROI, net profit after commission
- Also calls `updatePostSaleTracker()` for Post-Sale Tracker (34 columns)

K) Campaign Launch

`launchCampaign(dealIds, campaignType, campaignName)`:

1. Generates Campaign ID
2. For each deal: looks up seller contact info
3. Creates follow-up sequence entries in Campaign Queue
4. Schedules SMS/Email touches per sequence type
5. Returns campaignId

`launchCampaignUI()` (quick launch):

- Gets top 50 deals via `getTopDeals()`
- Filters to hot deals (verdict contains ■ or ■)
- Launches HOT_LEAD campaign on first 10 deals

CarHawk Ultimate — Browse.AI & Import Pipeline

Spec Pack Part 5 of 8 | QUANTUM-2.0.0

A) Architecture Overview

Source Files: quantum_browse_ai.gs (421 lines), quantum_browse_ai_api.gs (979 lines), quantum_robots.gs (1584 lines), quantum_import.gs (604 lines)

Total: 3,588 lines -- the largest subsystem in the platform.

Import Flow:

```
Browse.AI Robot (cloud)
↓ scrapes marketplace
Export Sheet (Google Sheet shared by Browse.AI)
↓
importFromBrowseAI()
↓
processBrowseAIIntegration()
↓ platform-aware column mapping
mapBrowseAIColumnsPlatform() + resolveField()
↓ platform-specific extras
extractPlatformExtras() + buildEnhancedDescription()
↓
Master Import Sheet (19 columns, raw data)
↓
quantumImportSync() (quantum_import.gs)
↓ parse, enrich, score
Master Database Sheet (61 columns, enriched)
```

API Direct Flow (Alternative):

```
Browse.AI v2 API
↓
runBrowseAITask() or runBrowseAIBulkRun()
↓
Browse.AI processes URLs in cloud
↓
doPost() webhook callback
↓
importWebhookData()
↓
Master Import Sheet
```

B) Supported Platforms

Automotive (6 platforms)

Platform	URL Patterns	Seller Detection	Extra Fields	Pagination
Facebook Marketplace	facebook.com/marketplace, fb.com/marketplace	dealer/private keywords	--	Infinite scroll
Craigslist	{city}.craigslist.org	dealer/private keywords	cylinders, modelSpecific	Click-next
OfferUp	offerup.com	dealer/private keywords	sellerRating, sellerJoined	Standard

Platform	URL Patterns	Seller Detection	Extra Fields	Pagination
eBay Motors	ebay.com (cat 6001)	dealer/private keywords	bidCount, watchers, endDate, listingType, sellerFeedback, shippingCost	Standard
AutoTrader	autotrader.com	dealer/private keywords	trim, mpg, features, accidents, owners, dealerRating, certifiedPreOwned, priceHistory	Standard
Cars.com	cars.com	dealer/private keywords	trim, features, dealValue, accidents, owners, dealerRating, homeDelivery	Standard

Powersports (4 platforms)

Platform	URL Patterns	Extra Fields
ATV Trader	atvtrader.com	hours, engineSize, vehicleType, features
Cycle Trader	cycletrader.com	hours, engineSize
Tractor House	tractorhouse.com	hours, engineSize, vehicleType, features, serialNumber
Craigslist ATVs/Motorcycles	craigslist.org/ata, /mca	cylinders, modelSpecific

---

C) Robot Registry (ROBOT_REGISTRY)

File: quantum_robots.gs (1584 lines)

Each platform entry contains:

```
{
platform: 'Facebook',
displayName: 'Facebook Marketplace',
category: 'automotive',
urlPatterns: [/regex1/, /regex2/, ...],
searchConfig: {
baseUrl: 'https://...',
locationFormat: 'city-state',
defaultRadius: 100,
defaultMinPrice: 500,
defaultMaxPrice: 25000,
defaultMinYear: 2010,
sortBy: 'creation_time_descend',
refreshInterval: 60, // minutes
maxPages: 5,
searchKeywords: ['cars', 'trucks', ...]
},
columnMap: {
url: ['url', 'link', 'listing_url', 'ad_url', ...],
title: ['title', 'name', 'listing_title', 'vehicle', ...],
price: ['price', 'asking_price', 'cost', ...],
// 15-20+ fields, each with multiple header variants
},
fieldDefaults: {
images: 0,
condition: 'Unknown',
sellerType: 'Unknown'
},
sellerDetection: {
```

```
dealerKeywords: ['dealership', 'dealer', 'auto sales', ...],
privateKeywords: ['private seller', 'my car', 'selling my', ...]
},
listingIdPattern: /regex/,
trainingGuide: {
startUrl: 'https://...',
robotType: 'List + Detail',
steps: ['Step 1: ...', 'Step 2: ...'],
fieldTraining: { title: 'CSS selector hint', ... },
paginationMethod: 'Infinite Scroll | Click Next',
importantNotes: ['Note 1', 'Note 2']
}
}
```

Craigslist Regional Support

9 subdomains configured:

- stlouis, kansascity, springfieldmo, columbiamo
- chicago, indianapolis, memphis, nashville, louisville

Categories: cta (all), cto (by owner), ctd (by dealer), ata (ATVs), mca (motorcycles), rva (RVs)

Column Map Field Variants

Each field has 5-20 header name variants to handle different Browse.AI export formats:

Field	Example Variants
url	url, link, listing_url, ad_url, marketplace_url, facebook_url, fb_link
title	title, name, listing_title, vehicle, vehicle_title, listing_name
price	price, asking_price, cost, listed_price, amount
mileage	mileage, miles, odometer, km, vehicle_mileage
condition	condition, vehicle_condition, item_condition
vin	vin, vin_number, vehicle_vin

D) Robot Registration & Management

Two registration methods:

Method 1: Sheet-Based (Legacy)

- Browse.AI exports data to a shared Google Sheet
- User registers the sheet ID + platform via `registerRobotUI()`
- System reads from export sheet during import
- Stored in Integrations sheet

Method 2: API-Based (Preferred)

- User provides Browse.AI API key via `setBrowseAIApiKeyUI()`
- Links robots from API via `linkBrowseAIRobotUI()`
- System deploys robots with bulk URLs and webhooks
- Tasks run in Browse.AI cloud, results fetched via API or webhook

Registration Functions

Function	Purpose
`registerBrowseAIRobot(platform, sheetId, robotName)`	Register sheet-based robot
`registerRobotUI()`	Interactive setup wizard
`linkBrowseAIRobotUI()`	API-based robot linking wizard
`showRobotSetupGuide()`	Display training instructions
`showRobotStatusUI()`	Show all registered robots and status
`listRegisteredRobots()`	Return robot list from Integrations sheet
`getSupportedPlatforms()`	Return categorized platform list

E) Browse.AI v2 API Integration

File: quantum_browse_ai_api.gs (979 lines)

API Configuration

```
const BROWSE_AI_API = {
BASE_URL: 'https://api.browse.ai/v2',
ENDPOINTS: {
ROBOTS: '/robots',
TASKS: '/robots/{robotId}/tasks',
TASK: '/robots/{robotId}/tasks/{taskId}',
BULK_RUN: '/robots/{robotId}/bulk-runs',
WEBHOOKS: '/robots/{robotId}/webhooks'
}
}
```

Authentication: Bearer token (API key stored in script properties)

Core API Functions

Function	Method	Purpose
`browseAIRequest(method, endpoint, payload)`	*	Core HTTP client with auth
`listBrowseAIRobots()`	GET	List all robots in account
`getBrowseAIRobot(robotId)`	GET	Get specific robot details
`runBrowseAITask(robotId, url, params)`	POST	Run single scrape task
`getBrowseAITask(robotId, taskId)`	GET	Get task status/results
`listBrowseAITasks(robotId, options)`	GET	List tasks with filters
`runBrowseAIBulkRun(robotId, urls, title)`	POST	Run bulk scrape (1000 max per batch)
`registerBrowseAIWebhook(robotId, url, event)`	POST	Register completion webhook

Bulk Run Batching

`runBrowseAIBulkRun()` splits URL lists into chunks of 1000:

- Each batch sent as separate API call
- 1-second pause between batches to avoid rate limits
- Returns array of bulk run IDs

Webhook Receiver

`doPost(e)` -- Google Apps Script web app endpoint:

1. Receives POST from Browse.AI on task completion
2. Extracts robotId, taskId, capturedTexts
3. Calls `importWebhookData()` to write to Master Import

F) Search URL Builders

Generate marketplace search URLs for bulk robot deployment:

buildFacebookMarketplaceURLs(params)

- Builds paginated Facebook search URLs
- Filters: minPrice, maxPrice, minYear, radius
- Returns array of URLs

buildCraigslistURLs(params)

- Builds across multiple regional subdomains
- Supports categories: cta, cto, ctd, ata, mca
- Multiple regions per call

buildOfferUpURLs(params)

- Category 7 (Vehicles)
- ZIP-based with radius filter

buildEbayMotorsURLs(params)

- Category 6001 (Cars & Trucks)
- Separate URLs for Buy It Now and Auction

buildAutoTraderURLs(params)

- Used vehicles filter
- Pagination support

buildCarsComURLs(params)

- Used stock filter
- Pagination support

buildATVTraderURLs(params) / buildCycleTraderURLs(params)

- Price and radius filters
- Pagination support

buildCraigslistATVURLs(params)

- ATVs/UTVs/Snowmobiles and Motorcycles/Scooters categories

G) Robot Deployment

Single Platform Deploy

`deployMarketplaceRobot(platform, robotId, searchParams)`:

1. Generates search URLs via appropriate builder
2. Runs bulk scrape via `runBrowseAIBulkRun()`

3. Registers webhook for completion notifications
4. Returns `{ platform, robotId, urlCount, bulkRuns }`

Deploy All

`deployAllRobots(searchParams)`:

- Iterates all registered robots in Integrations sheet
- Deploys each one via `deployMarketplaceRobot()`

H) Import Processing (Sheet-Based)

File: quantum_browse_ai.gs (421 lines)

`importFromBrowseAI()` -- Main Entry Point

1. Fetches all active Browse.AI integrations from Integrations sheet
2. For each integration:
- Opens the export sheet by ID
- Reads all data rows
- Maps columns based on platform via `mapBrowseAIColumnsPlatform()`
- For each row:
- Skips if URL already processed (deduplication via script properties)
- Resolves fields using `resolveField()` (primary + fallbacks)
- Extracts platform-specific extras
- Builds enhanced description with [TAG: value] annotations
- Builds enhanced seller info with type detection
- Appends row to Master Import sheet
- Marks URL as processed
3. Updates integration sync status
4. Shows summary alert

Platform-Specific Extras

`extractPlatformExtras()` extracts fields unique to each platform:

Platform	Extra Fields
eBay	bidCount, watchers, endDate, listingType, sellerFeedback, shippingCost
AutoTrader	trim, mpg, features, accidents, owners, dealerRating, certifiedPreOwned, priceHistory
Cars.com	trim, features, dealValue, accidents, owners, dealerRating, homeDelivery
OfferUp	sellerRating, sellerJoined
Craigslist	cylinders, modelSpecific
Powersports	hours, engineSize, vehicleType, features, serialNumber

Enhanced Description Tags

`buildEnhancedDescription()` embeds structured data:

```
[Original description text]
[MILEAGE: 45000]
[VIN: 1HGCV1F34LA000001]
[CONDITION: Good]
```

```
[TRANSMISSION: Automatic]
[FUEL_TYPE: Gasoline]
[BODY_STYLE: Sedan]
[EXTERIOR_COLOR: Silver]
[DRIVETRAIN: FWD]
[TITLE_STATUS: Clean]
```

URL Deduplication

- `getProcessedUrls()` reads from PropertiesService (max 1000 URLs)
- `markUrlProcessed(url)` adds to stored list
- Prevents re-importing same listings

I) Import Sync to Master Database

File: quantum_import.gs (604 lines)

`quantumImportSync()` -- Processes Master Import rows into Master Database:

1. Reads unprocessed rows from Master Import (Processed = false)
2. For each row:
   - Parses raw fields (title → year/make/model, price → number, etc.)
   - Calls `calculateQuantumMetrics(parsed)` for all scoring
   - Generates Deal ID
   - Writes 61-column row to Master Database
   - Calls `addToLeadsTracker()` to create lead entry
   - Marks import row as processed (col 16 = true, col 17 = Deal ID)
3. Shows processing dialog with count

J) Seller Type Detection

File: quantum_robots.gs

`detectSellerType(sellerInfo, description, platform)`:

Dealer Keywords: dealership, dealer, auto sales, motors, automotive, car lot, pre-owned, certified, inventory, financing available, we finance, buy here pay here, bhph

Private Keywords: private seller, private owner, personal vehicle, my car, selling my, owner, one owner, clean title

Returns: 'Dealer', 'Private', or 'Unknown'

K) UI Functions for Browse.AI

Function	UI Type	Purpose
`setBrowseAIApiKeyUI()`	Prompt	Enter API key (stored in script properties)
`showBrowseAIRobotsUI()`	Dialog	List all robots from API
`linkBrowseAIRobotUI()`	Wizard	Link API robot to platform
`registerRobotUI()`	Wizard	Register sheet-based robot
`showRobotSetupGuide()`	Modal	Display training instructions
`deployRobotUI()`	Wizard	Deploy robot(s) with URL generation
`fetchAndImportBrowseAIDataUI()`	Alert	Fetch API task results
`showRobotStatusUI()`	Modal	All robots and platforms status

CarHawk Ultimate — Integrations & External APIs

Spec Pack Part 6 of 8 | QUANTUM-2.0.0

A) Integration Architecture

Source Files: quantum_integrations.gs (93 lines), quantum_sms_it.gs (175 lines), quantum_ohmylead.gs (83 lines), quantum_companyhub.gs (237 lines), quantum_crm_api.gs (139 lines)

All integrations are tracked in the **Integrations sheet** (20 columns) and managed via CRUD functions in quantum_integrations.gs.

B) Integration Registry

`getActiveIntegrations()`

Returns array of active integration objects from Integrations sheet:

```
{
integrationId: string,
provider: string, // Browse.ai, SMS-iT, Ohmylead, CompanyHub
type: string, // Robot, Sheet, API, Webhook
name: string,
key: string, // API key
secret: string,
status: string, // Active, Inactive
lastSync: DateTime,
configuration: object, // JSON parsed
webhookUrl: string,
notes: string
}
```

`addIntegration(data)`

Creates new record in Integrations sheet with auto-generated ID.

`updateIntegrationSync(id, recordsSynced)`

Updates last sync timestamp and accumulates records synced count.

`updateIntegrationError(id, error)`

Increments error count and stores last error message.

C) SMS-iT Integration

File: quantum_sms_it.gs (175 lines)

Purpose: Export hot deals as leads to SMS-iT for campaign management.

Export Flow

```
Master Database
↓ filter: verdict = '■ HOT DEAL' or '■ SOLID DEAL' AND phone exists
exportQuantumSMS()
↓ show preview dialog
getQuantumSMSExportHTML(hotDeals)
↓ user selects deals, writes message, configures campaign
```

```
processQuantumSMSExport(config)
↓ format data, send webhook
sendToSMSIT(data, webhookUrl)
↓
SMS-iT Platform → SMS delivery
↓
logCRMExport() + createFollowUpSequence() (optional)
```

SMS-iT Webhook Payload

```
POST {SMSIT_WEBHOOK_URL}
{
source: 'CarHawk Ultimate',
version: QUANTUM.VERSION,
timestamp: ISO string,
leads: [
{
dealId: string,
phone: '+1XXXXXXXXXX',
name: string,
customFields: {
vehicle: '2020 Honda Civic',
year: 2020,
make: 'Honda',
model: 'Civic',
price: 15000,
platform: 'Facebook',
verdict: '■ HOT DEAL',
roi: 45,
profit: 3000,
distance: 12
},
tags: ['carhawk', 'hot-deal', 'facebook', 'contacted'],
message: string,
campaignName: string,
sequenceType: 'HOT_LEAD'
}
]
}
```

Sequence Type Classification

`determineSequenceType(dealData)`:

Condition	Sequence
Verdict contains ■, confidence > 85	HOT_LEAD
Verdict contains ■, days listed < 14	WARM_LEAD
Everything else	COLD_LEAD

Database Columns Referenced

Col Index	Field
0	Deal ID
2	Platform
5-7	Year, Make, Model
13	Price
16	Distance
26-27	Profit, ROI

Col Index	Field
28	Flip Strategy
32	Days Listed
33-34	Seller Name, Phone
40	AI Analysis Message
41-42	Confidence, Verdict

D) Ohmylead Integration

File: quantum_ohmylead.gs (83 lines)

Purpose: Two-way appointment sync with Ohmylead booking platform.

Outbound Sync

`syncOhmyleadAppointments()`:

1. Reads Appointments sheet
2. Filters Status = "Scheduled" AND synced flag (col 17) = false
3. Sends each appointment to Ohmylead webhook
4. Marks as synced on success

Payload sent:

```
POST {OHMYLEAD_WEBHOOK_URL}
{
appointmentId: string,
dealId: string,
vehicle: string,
sellerName: string,
phone: string,
scheduledTime: DateTime,
location: string,
notes: string
}
```

Inbound Booking

`receiveOhmyleadBooking(bookingData)`:

1. Receives booking from Ohmylead webhook callback
2. Creates appointment via `scheduleAppointment()`
3. Updates deal stage to APPOINTMENT_SET
4. Logs SMS conversation if booking originated from SMS

Booking data received:

```
{
dealId: string,
scheduledTime: DateTime,
location: string,
locationType: 'In-Person',
duration: 30,
type: 'Viewing',
notes: string,
source: 'SMS' (optional)
}
```

E) CompanyHub CRM Export

File: quantum_companyhub.gs (237 lines)

Purpose: Export recommended deals to CompanyHub CRM via CSV file on Google Drive.

Export Flow

```
Master Database
↓ filter: Recommended = 'YES' (col 44)
exportQuantumCRM()
↓ user confirms
formatForCompanyHub(deals)
↓ map stages, calculate probability
generateCompanyHubCSV(data)
↓ save to Google Drive
logCRMExport()
```

Stage Mapping

CarHawk Stage	CompanyHub Stage
IMPORTED	New Lead
CONTACTED	Contacted
RESPONDED	Qualified
APPOINTMENT_SET	Meeting Scheduled
NEGOTIATING	Negotiation
CLOSED_WON	Closed Won
LOST	Closed Lost

Deal Probability Calculation

`calculateDealProbability(deal)`:

Base probability by verdict:

Verdict	Base %
■ HOT DEAL	80%
■ SOLID DEAL	60%
■■ PORTFOLIO FOUNDATION	40%
■ PASS	5%

Adjustments:

- +20% if stage = APPOINTMENT_SET
- +10% if stage = RESPONDED
- +10% if response rate > 80%
- Capped at 5-95%

Expected Close Date

`calculateExpectedCloseDate(deal)`:

- Uses knowledge base avg days to sell
- HOT DEAL: 70% of base time
- PASS: 200% of base time
- Returns ISO date string

Tag Generation

`generateCompanyHubTags(deal)`:

Tags applied based on deal attributes:

- Verdict: hot-deal, solid-deal
- Platform: facebook, craigslist, etc.
- Distance: local (<25mi), regional (25-75mi), distant (>75mi)
- Financial: high-roi (>50%), high-profit (>$3000)
- Engagement: contacted, responsive

CSV Export Fields

Field	Source
Company	Seller Name
Contact Name	Seller Name
Phone	Seller Phone
Email	Seller Email
Deal Name	"Year Make Model"
Deal Value	Price
Expected Profit	Profit
ROI %	ROI
Stage	Mapped stage
Probability	Calculated
Expected Close Date	Calculated
Lead Score	Quantum Score
Source	Platform
Location	Location
Distance	Distance
Days on Market	Days Listed
Contact Attempts	Contact Count
Response Rate	Response Rate
Tags	Generated tags
CarHawk ID	Deal ID
Verdict	Verdict
Vehicle	"Year Make Model"

Turo fields (if module enabled):

- Turo Score, Turo Monthly Net, Turo Payback Months
- Turo Risk Tier, Turo Status, Fleet ID

F) API Delivery Layer

File: quantum_crm_api.gs (139 lines)

SMS Delivery

Function	Service	Priority
`sendViaSMSIT(phone, message, dealId)`	SMS-iT	1st (webhook then API)

Function	Service	Priority
`sendSMS(phone, message)`	Twilio	2nd (fallback)

Phone formatting: `formatPhoneForSMSIT()` → +1XXXXXXXXXX

Email Delivery

Function	Service	Priority
`sendEmail(to, subject, body)`	SendGrid	1st
(fallback)	Google MailApp	2nd

Combined Functions

Function	Purpose
`sendFollowUpSMS(dealId, templateName)`	Look up deal, fill template, send SMS
`sendFollowUpEmail(dealId, templateName)`	Look up deal, fill template, send email
`sendReminderSMS(phone, message)`	Direct SMS for appointment reminders
`sendCampaignSMS(phone, message)`	Campaign SMS via SMS-iT
`sendCampaignEmail(email, subject, msg)`	Campaign email via SendGrid

G) Required API Credentials

All stored in the Settings sheet:

Service	Required Keys	Purpose
OpenAI	OPENAI_API_KEY	AI deal analysis
Browse.AI	(script properties)	Web scraping
SMS-iT	SMSIT_API_KEY, SMSIT_WEBHOOK_URL	SMS campaigns
Ohmylead	OHMYLEAD_WEBHOOK_URL	Appointment sync
Twilio	TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE	Fallback SMS
SendGrid	SENDGRID_API_KEY	Email delivery
CompanyHub	(file export, no API key)	CRM export

Minimum required: OpenAI API key, Alert Email

Recommended: + SMS-iT for outreach

Full CRM: + Ohmylead + SendGrid/Twilio

H) Google Services Used

Service	Scope	Purpose
SpreadsheetApp	spreadsheets	All sheet operations
DriveApp	drive	CSV exports, file storage
UrlFetchApp	script.external_request	All external API calls
MailApp	gmail.send	Fallback email delivery
ScriptApp	script.scriptapp	Trigger management

Service	Scope	Purpose
PropertiesService	(built-in)	URL deduplication, API key storage
HtmlService	script.container.ui	Dialogs, modals, sidebars

I) Sync Status Tracking

Every integration sync is tracked in the Integrations sheet:

Column	Purpose
Last Sync	Timestamp of last successful sync
Next Sync	Calculated from sync frequency
Sync Frequency	Minutes between syncs
Records Synced	Cumulative count
Error Count	Number of failures
Last Error	Most recent error message
Status	Active / Inactive

The `syncQuantumCRM()` menu handler checks each configured integration and runs its sync function:

- SMS-iT: `exportQuantumSMS()`
- Ohmylead: `syncOhmyleadAppointments()`

CarHawk Ultimate — UI Components & HTML Dialogs

Spec Pack Part 7 of 8 | QUANTUM-2.0.0

A) UI Architecture

Source Files: quantum_ui.gs (224 lines), quantum_menu_handlers.gs (538 lines), quantum_setup_html.gs (309 lines), quantum_processing_html.gs (87 lines), quantum_sms_export_html.gs (322 lines), plus 18 standalone HTML files.

Design System:

- Dark theme: #1a1a2e background, #00d4ff cyan accent, #e0e0e0 text
- Font: Segoe UI, Arial, sans-serif (monospace for VIN input)
- Card-based layouts with consistent styling
- Status badges with color coding
- No external CSS/JS libraries -- all custom inline
- All dialogs use `google.script.host.close()` for dismissal

B) Server-Side Generated HTML (3 files)

Setup Wizard (quantum_setup_html.gs -- 309 lines)

Property	Value
Type	Modal dialog
Size	800 x 600
Function	`getQuantumSetupHTML()`
Launched by	`initializeQuantumSystem()`

Features:

- Glassmorphism container with backdrop blur
- Pulsing quantum logo animation
- 8-item feature grid: AI Analysis, Market Data, Scoring, Workflows, Analytics, Alerts, SMS-iT, Ohmylead
- Configuration form: Business Name, Home ZIP, OpenAI API Key, SMS-iT Key, Ohmylead Webhook, Profit Target, Analysis Depth, Alert Email
- Calls `google.script.run.deployQuantumArchitecture(config)` on submit
- Loading overlay with spinner and status updates

Processing Dialog (quantum_processing_html.gs -- 87 lines)

Property	Value
Type	Modal dialog
Function	`getQuantumProcessingHTML(count)`

Features:

- Gradient background (#667eea to #764ba2)
- Animated spinner (rotating border)
- Animated progress bar (gradient pink-red-purple)

- Stats text: "Processing X items..."

SMS Export Dialog (quantum_sms_export_html.gs -- 322 lines)

Property	Value
Type	Modal dialog
Function	`getQuantumSMSExportHTML(hotDeals)`
Launched by	`exportQuantumSMS()`

Features:

- 3-card stats bar: Total Leads, Total Value, Avg ROI
- Campaign settings form: name, message template, send delay, include AI analysis, create follow-ups
- Deal selection table with select all/none toggle, checkboxes per row
- Columns: Deal ID, Vehicle, Price, ROI %, Seller Name, Phone, Verdict
- Live message preview with placeholder interpolation ({name}, {year}, {make}, {model}, {price})
- Calls `google.script.run.processQuantumSMSExport(config)` on export

C) Standalone HTML Components (18 files)

Appointments & Scheduling

File	Type	Size	Launcher	Dimensions
AppointmentManager.html	Modal	68 lines	`openAppointmentManager()`	800 x 600

- Today's appointments list with time, customer, vehicle, type
- Color-coded badges: test-drive, delivery, inspection
- New Appointment button

CRM & Communication

File	Type	Size	Launcher	Dimensions
CallLogs.html	Modal	88 lines	`openCallLogs()`	800 x 600
SMSConversations.html	Modal	86 lines	`openSMSConversations()`	800 x 600
CampaignManager.html	Modal	78 lines	`openCampaignManager()`	800 x 600
FollowUpCenter.html	Modal	29 lines	`openFollowUpCenter()`	800 x 600
FollowUpSequences.html	Modal	64 lines	`manageFollowUpSequences()`	800 x 600

CallLogs.html:

- 4-metric grid: Total Calls (47), Outbound (28), Inbound (15), Missed (4)
- Call log entries with direction icons (green inbound, cyan outbound, red missed)
- Duration, customer, purpose, timestamp

SMSConversations.html:

- Conversation list with 4 contacts, avatar initials, last message preview, unread indicator
- Expandable chat view with incoming/outgoing message bubbles and timestamps

CampaignManager.html:

- Campaign list: name, channel (SMS/Email), start date, stats (sent, open rate, reply rate)

- Status badges: Active, Completed, Draft

FollowUpCenter.html:

- Active follow-ups and scheduled actions sections (stub for expansion)

FollowUpSequences.html:

- Active sequences with step counts and contact counts
- Detailed 5-step sequence view: Welcome SMS (immediate), Follow-up Call (1hr), Vehicle Details Email (1 day), Check-in SMS (3 days), Final Offer (7 days)
- Status badges: Active, Paused

Deal Analysis & Calculators

File	Type	Size	Launcher	Dimensions
DealAnalyzer.html	Modal	90 lines	`showDealAnalyzer()`	800 x 600
DealCalculator.html	Modal	79 lines	`openDealCalculatorPro()`	600 x 800 (sidebar)
DealGallery.html	Full Page	115 lines	`showDealGallery()`	1200 x 800
VINDecoder.html	Modal	81 lines	`openQuantumVINDecoder()`	600 x 500

DealAnalyzer.html:

- Form: Platform (dropdown from server), Vehicle, Listing Price, Market Value, Notes
- Client-side analysis: spread, margin %, risk score (Low/Medium/High)
- Results hidden until "Analyze Deal" clicked
- Template injection: `<?!= JSON.stringify(platforms || []) ?>`

DealCalculator.html:

- Real-time calculator with oninput triggers
- Inputs: Purchase Price, Repair/Reconditioning, Transport/Fees, Expected Sale Price
- Live results: Total Cost, Gross Profit, Margin %, Net Profit
- Color-coded profit (green positive, red negative)

DealGallery.html:

- Stats bar: Total Deals, Hot Deals, Total Profit Potential, Avg ROI
- Auto-fill grid of deal cards (280px min width)
- Card fields: Year/Make/Model, Platform, Location, Price, Profit, ROI, Mileage, Verdict
- Verdict badges: red (HOT), green (SOLID), gray (PASS)
- Action buttons: View, Analyze, Contact
- Server calls: `<?!= JSON.stringify(deals || []) ?>`, `google.script.run.processDealAction(action, dealId)`

VINDecoder.html:

- 17-character VIN input with live character counter
- Monospace font, uppercase transformation
- Results card: Year, Make, Model, Engine, Drive Type, Plant, Country
- Server call: `google.script.run.decodeVIN(vin)`

Analytics & Pipeline

File	Type	Size	Launcher	Dimensions
CRMAnalytics.html	Modal	85 lines	`openCRMAnalytics()`	1000 x 700
PipelineView.html	Modal	54 lines	`openPipelineView()`	1200 x 800
SpeedToLead.html	Modal	51 lines	`openSpeedToLead()`	600 x 800

CRMAnalytics.html:

- 4-metric KPI grid: Revenue MTD ($142K), Deals Closed (23), Close Rate (34%), Avg Deal Profit ($6.2K)
- Change indicators (up/down percentages)
- Horizontal bar chart: deals by source (Facebook, Website, Referral, Walk-in, Other)
- Sales funnel: 85 leads → 62 contacted → 41 qualified → 29 negotiating → 23 closed

PipelineView.html:

- Kanban-style 4-column pipeline: New Leads (3), Contacted (2), Negotiating (1), Closed Won (1)
- Deal cards with customer name, vehicle, price
- Horizontal scroll layout

SpeedToLead.html:

- 3 metric cards: Avg Response (2.4 min), Under 5 Min (87%), Today's Leads (14)
- Lead response time list, color-coded: Fast (green <5min), Medium (yellow 5-10min), Slow (red >10min)

System & Configuration

File	Type	Size	Launcher	Dimensions
Settings.html	Sidebar	90 lines	`openQuantumSettings()`	600 x 800
IntegrationManager.html	Modal	87 lines	`openIntegrationManager()`	800 x 600
KnowledgeBase.html	Modal	57 lines	`openKnowledgeBase()`	800 x 600
QuantumHelp.html	Modal	56 lines	`showQuantumHelp()`	900 x 700
QuickActions.html	Sidebar	46 lines	`showQuickActions()`	350 (sidebar)

Settings.html:

- General toggles: Auto-Sync Inventory (on), Email Notifications (on), SMS Notifications (off)
- API Configuration: Browse.ai API Key, SMS-iT API Key inputs
- Thresholds: Min Profit Target ($1500), Speed-to-Lead Goal (5 min)
- Custom toggle switch components

IntegrationManager.html:

- 5 integration cards: Browse.ai (Connected), SMS-iT (Connected), VoIP (Setup Required), Email (Disconnected), Payment Gateway (Disconnected)
- Status indicators: green/red/yellow

KnowledgeBase.html:

- Search box for vehicles, guides, market data
- 4 reference cards: Common Recalls (2020-2024), Pricing Guide: Trucks & SUVs, Title Status Explained, EV Battery Health
- Tag system for categorization

QuantumHelp.html:

- 4 guide cards: Getting Started, Deal Workflow, Inventory Management, Analytics & Reports
- Keyboard shortcuts: Ctrl+Q (Quick Actions), Ctrl+N (New Deal), Ctrl+K (Search), Ctrl+S (Sync)

QuickActions.html:

- 4 action buttons: Sync Inventory, Analyze Deals, Check Alerts, Export Data
- Server call: `google.script.run.handleQuickAction(action)`

D) Server-Side UI Functions

File: quantum_ui.gs (224 lines)

Data Retrieval

Function	Purpose	Returns
`getTopDeals(limit=20)`	Top deals from DB (verdict != PASS)	Array of deal objects
`getQuickStats()`	Aggregate stats from top 100 deals	`{ totalDeals, hotDeals, totalProfit, avgROI, platforms }`

Action Routing

`processDealAction(action, dealId)`:

Action	Routes To
'view'	`navigateToDeal()` -- scrolls to deal row
'export'	`exportSingleDeal()` -- SMS export single deal
'analyze'	`analyzeSingleDeal()` -- AI analysis
'contact'	`initiateDealContact()` -- HOT_LEAD follow-up
'schedule'	`scheduleDealAppointment()` -- open scheduler

`runQuickAction(action)`:

Action	Routes To
'sync'	`runQuantumSync()` -- Browse.AI + import
'analyze'	`executeQuantumAIBatch()` -- batch AI
'alerts'	Returns alert count
'export'	`exportQuantumSMS()` -- hot deal export

E) System Diagnostics

`runSystemDiagnostics()` checks 5 health categories:

Check	Function	Healthy When
Sheets	`checkSheets()`	All QUANTUM_SHEETS exist
Settings	`checkSettings()`	Settings sheet has rows
Integrations	`checkIntegrations()`	Active integrations > 0
Triggers	`checkTriggers()`	Project triggers >= 3
Performance	`checkPerformance()`	Sheet response < 3 seconds

Returns overall health status report via alert dialog.

CarHawk Ultimate — Triggers, Alerts & Reporting

Spec Pack Part 8 of 8 | QUANTUM-2.0.0

A) Trigger System

Source File: quantum_triggers.gs (101 lines)

Trigger Deployment

`deployQuantumTriggers()`:

1. Clears all existing project triggers via `ScriptApp.deleteTrigger()`
2. Creates 3 trigger groups:

Trigger	Frequency	Function	Purpose
Hourly Sync	Every 1 hour	`quantumHourlySync()`	Import + data refresh
Daily Analysis	Daily at 6 AM	`quantumDailyAnalysis()`	Batch AI + reports
CRM Triggers	Variable	`setupCRMTriggers()`	Follow-ups + campaigns

CRM Sub-Triggers (from quantum_crm_automation.gs)

Trigger	Frequency	Function
Follow-Up Processor	Every 5 minutes	`processFollowUps()`
Campaign Processor	Every 10 minutes	`processCampaigns()`
Appointment Reminders	Every 1 hour	`processAppointmentReminders()`

B) Hourly Sync

`quantumHourlySync()`:

Condition: Only runs if `REALTIME_MODE` setting = true

Steps:

1. `runBrowseAIIntegrations()` -- Fetch data from all Browse.AI robots
2. `runImportSync()` -- Process import queue to Master Database
3. Update `LAST_SYNC` setting with current timestamp

Skips execution entirely if realtime mode is disabled.

C) Daily Analysis (6 AM)

`quantumDailyAnalysis()`:

Steps (always run):

1. `executeQuantumAIBatch()` -- AI analysis on all unanalyzed deals
2. `generateQuantumDashboard()` -- Refresh dashboard metrics
3. `checkQuantumAlerts()` -- Process alert queue

Conditional steps:

4. `syncCRMData()` -- Only if `CRM_ENABLED` = true
5. `batchAnalyzeTuro()` -- Only if Turo Engine sheet exists
6. `checkComplianceAlerts()` -- Only if Turo Engine sheet exists

Post-run:

7. Update `LAST_ANALYSIS` setting with current timestamp
8. Log completion to Activity Logs

D) Alert System

Source File: quantum_alerts.gs (272 lines)

Alert Conditions

`checkQuantumAlerts(dealData, analysis)` evaluates 3 alert types:

Alert Type	Condition	Priority
HOT_DEAL	verdict = '■ HOT DEAL' AND confidence > 85%	URGENT
HIGH_PROFIT	profitPotential > PROFIT_TARGET setting	HIGH
QUICK_FLIP	strategy = 'Quick Flip' AND quickSaleProbability > 80%	MEDIUM

Alert Object Structure

```
{
type: 'HOT_DEAL' | 'HIGH_PROFIT' | 'QUICK_FLIP',
priority: 'URGENT' | 'HIGH' | 'MEDIUM',
title: string,
message: string,
actions: ['View Deal', 'Contact Seller', 'Export to CRM']
}
```

Alert Processing Pipeline

```
Deal analyzed
↓
checkQuantumAlerts() → evaluate 3 conditions
↓
processQuantumAlerts() → log to Activity Logs
↓ if REALTIME_ALERTS = true
sendQuantumNotifications() → filter URGENT alerts
↓ if ALERT_EMAIL configured
sendQuantumAlertEmail()
```

Alert Email Format

HTML email with:

- Gradient header with ■■ Quantum Alert branding
- Alert cards (URGENT = red background, others = blue)
- Metrics display: Asking Price, Distance, Days Listed
- CTA button linking to spreadsheet
- Footer with business name

Alert Digest

`sendQuantumDigest()`:

- Collects all queued alerts from `QuantumState.realTimeAlerts`

- Sends as single digest email
- Clears queue after sending

E) Reporting System

Source File: quantum_reports.gs (196 lines)

Closed Deals Report

`generateClosedDealsReport()`:

- Reads Closed Deals sheet
- Creates summary report sheet with:

Metric	Source
Total Closed Deals	Row count
Total Profit	Sum of column 14 (Profit)
Average Profit	Total / count
Average ROI	Average of column 15 (ROI %)
Average Days to Close	Average of column 12

Weekly Report

`generateQuantumWeekly()`:

- Creates "Weekly Report" sheet with metrics for the past 7 days:

Metric	Calculation
Week Ending	Current date
Deals Analyzed	Deals imported in past 7 days
Deals Contacted	Contact count > 0
Appointments Set	Stage = APPOINTMENT_SET
Deals Closed	Stage = CLOSED_WON
Total Profit	Sum of closed deal profits

Monthly Report

`generateQuantumMonthly()`:

- Shows alert with monthly summary (placeholder for expanded implementation)

Performance Matrix

`generatePerformanceMatrix()`:

- Shows alert dialog with:

Metric	Value
Deals Analyzed	Queue length
Hot Deals Found	From `getQuickStats()`
Avg Response Time	< 5 minutes
Conversion Rate	23%
Avg Profit	$3,200

ROI Optimizer

`runROIOptimizer()`:

- Analyzes ROI patterns across all deals
- Shows alert with:
- Average ROI % across all deals
- High ROI deals (>50%) with vehicle list
- Low ROI deals (<20%) with common issues
- Recommendations:

1. Focus on vehicles matching high ROI patterns
2. Avoid vehicles with major repair issues
3. Target quick flip strategies

F) Dashboard System

Source File: quantum_dashboard.gs (202 lines)

`generateQuantumDashboard()` creates/refreshes a dashboard sheet with 5 sections:

Section 1: Header

- Title: "CarHawk Ultimate -- Quantum Dashboard"
- Subtitle with timestamp

Section 2: Key Metrics

Metric	Source	Icon
Total Deals	Row count in DB	■
Hot Deals	Col 42 count ("■ HOT DEAL")	■
Active Capital	Sum col 13 where status = Active	■
Avg ROI	Average of col 27	■
Quick Flips	Count col 29 = 'Quick Flip'	■

Section 3: Charts

- Chart placeholder section (for future chart implementation)

Section 4: Leaderboard

Category	Source
Top ROI Deal	Highest col 27 value
Highest Profit	Highest col 26 value
Fastest Flip	Placeholder (days)
Best Platform	Facebook (65% success)
Hot Streak	Current week count

Section 5: Market Insights

`generateMarketInsights()` returns dynamic insights:

Condition	Insight
SUV count > sedan count	"SUV demand trending higher than sedans"
Quick Flip count > 5	"Quick flip opportunities above average"
(always)	"Sweet spot: 2018-2020 models with under 80k miles"
(always)	"Risk alert: Avoid high-mileage luxury brands"
(always)	"Profit optimizer: Target vehicles 20-30% below market"

Dashboard Styling

Property	Value
Title Font	Google Sans, 24pt bold
Subtitle	Gray (#666666), 14pt
Metric Values	20pt
Column Widths	Col 1: 200px, Col 2-3: 150px
Borders	Solid #e0e0e0 on A5:J50

G) Testing System

Source File: quantum_testing.gs (152 lines)

Test Functions

Function	Purpose
`testCRMFunctions()`	Tests appointment, SMS, and campaign creation
`createTestDeal()`	Creates sample deal in database
`simulateInboundSMS()`	Logs 4 test SMS responses
`simulateCallLog()`	Creates test AI call entry
`simulateCampaignRun()`	Launches test campaign on 3 deals
`testBrowseAIImport()`	Tests Browse.AI integration setup

Test Deal Data

```
{
dealId: 'TEST-{timestamp}-{random}',
vehicle: '2020 Honda Civic EX',
color: 'Silver',
price: 15000,
mileage: 50000,
condition: 'Excellent',
conditionScore: 95,
platform: 'Test Platform',
location: 'St. Louis, MO',
zip: '63101',
daysListed: 10,
profit: 3000,
roi: 20
}
```

Simulated Inbound SMS Messages

#	Message	Expected Intent
1	"Yes it's still available"	AVAILABLE
2	"Sorry, already sold"	SOLD
3	"Can you do $8000?"	NEGOTIATION
4	"When can we meet?"	SCHEDULING

Simulated Call Data

- Duration: 5 minutes
- Direction: INBOUND
- Transcription mentions availability and appointment scheduling
- Expected outcome: Appointment Set

H) Activity Logging

All system events are logged to the Activity Logs sheet via two functions:

`logQuantum(action, details)`

- Level: INFO
- Category: SYSTEM
- Includes: timestamp, user email, success flag

`logCRMActivity(action, dealId, details)`

- Level: INFO
- Category: CRM
- Includes: timestamp, deal ID association, user email

Log Levels Used

Level	Used For
INFO	Normal operations, syncs, exports
ALERT	Hot deal alerts, compliance warnings
ERROR	API failures, sync errors

I) Utility Functions Reference

Source File: quantum_utilities.gs (392 lines)

Function	Purpose
`getQuantumSheet(name)`	Get sheet by name
`getQuantumSetting(key)`	Read setting from Settings sheet
`setQuantumSetting(key, value)`	Write/update setting
`logQuantum(action, details)`	Write to Activity Logs
`logCRMActivity(action, dealId, details)`	Write CRM event to logs
`generateQuantumId(prefix)`	Generate unique ID: PREFIX-timestamp-random
`standardizeMake(make)`	Normalize make names (Chevy → Chevrolet)

Function	Purpose
`detectPlatform(url)`	Detect platform from URL
`detectHotSeller(data)`	Flag urgent seller signals
`detectMultipleVehicles(desc)`	Flag dealer inventory
`formatTime(date)`	Format as "MMM dd, h:mm a"
`analyzeInboundResponse(dealId, msg)`	Process seller reply
`processCallInsights(dealId, callData)`	Analyze call transcript
`getEmailTemplate(name, deal)`	Get HTML email template
`updatePostSaleTracker(dealId, closeId, data)`	Write post-sale record
`updateExportRecordIds(leadIds)`	Update CRM export with IDs
`logCRMExport(platform, count, fileId)`	Log CRM export event
`include(filename)`	HTML template inclusion

Spec Pack Index

Part	File	Covers
1	SPEC_PACK_PART1_SYSTEM_OVERVIEW.md	Identity, architecture, file inventory, config, menu, initialization
2	SPEC_PACK_PART2_SHEET_SCHEMAS.md	All 25 core + Turo sheet schemas with exact column definitions
3	SPEC_PACK_PART3_AI_ENGINE.md	OpenAI integration, calculations, scoring, knowledge base
4	SPEC_PACK_PART4_CRM_ENGINE.md	CRM pipeline, follow-ups, SMS/email, automation, API layer
5	SPEC_PACK_PART5_BROWSE_AI.md	Browse.AI robots, import pipeline, 10 platforms, URL builders
6	SPEC_PACK_PART6_INTEGRATIONS.md	SMS-iT, Ohmylead, CompanyHub, API delivery, credentials
7	SPEC_PACK_PART7_UI_COMPONENTS.md	18 HTML dialogs, server-side HTML, design system
8	SPEC_PACK_PART8_TRIGGERS_ALERTS.md	Triggers, alerts, reporting, dashboard, testing, utilities
9	**SPEC_PACK_PART9_PAM.md**	**Project Arbitrage Module -- projects, needs, matching, AI evaluation**
Turo	TURO_SPEC_PACK.md	Complete Turo Rental Hold Module specification

CarHawk Ultimate — Project Arbitrage Module (PAM)

Spec Pack Part 9 of 9 | QUANTUM-2.0.0

A) Module Overview

The Project Arbitrage Module (PAM) extends CarHawk Ultimate with a project-based sourcing engine. Instead of analyzing individual vehicle deals, PAM lets you define **projects** (vehicle builds, property rehabs, ATV builds, etc.) with specific **needs** (parts, materials, components), then automatically **matches** those needs against scraped marketplace listings using a 2-layer scoring system: rule-based keyword matching (Layer 1) followed by OpenAI evaluation (Layer 2).

Architecture Role: Additive module -- zero breaking changes to existing CarHawk core. All PAM logic lives in 7 dedicated `.gs` files plus 1 HTML dashboard. The module reads from the existing Master Database (or any configured source sheet) and writes results to its own 6 sheets.

Workflow:

1. Create projects with budgets and timelines
2. Define needs per project (parts, materials, components with keywords and price targets)
3. Run Logic Engine -- scores every scraped listing against every open need
4. Run AI Evaluation -- OpenAI evaluates top matches for fit, strategy, and negotiation
5. Review matches in the Command Center dashboard
6. Record purchases, track budgets, monitor fulfillment

B) File Inventory

#	File	Lines	Purpose
1	PAM_Settings.gs	113	Settings sheet management and defaults
2	PAM_Utils.gs	154	ID generation, keyword scoring, formatting helpers
3	PAM_Sheets.gs	258	Idempotent sheet creation for 6 PAM sheets
4	PAM_DataAccess.gs	310	CRUD operations for all PAM sheets (UI data layer)
5	PAM_LogicEngine.gs	228	Layer 1 rule-based keyword + scoring engine
6	PAM_AIEngine.gs	258	Layer 2 OpenAI gpt-4o-mini evaluation
7	PAM_Menu.gs	61	Menu registration and test data seeder
8	PAM_UI.html	1,654	Full Command Center dashboard (5 tabs)
	Total	**3,036**	

C) Sheet Inventory

#	Sheet Name	Columns	Purpose
1	PAM_Projects	13 (A-M)	Project definitions with budgets
2	PAM_Needs	19 (A-S)	Item needs per project with keywords
3	PAM_Matches	25 (A-Y)	Scored listing-to-need matches
4	PAM_Purchases	15 (A-O)	Recorded purchases
5	PAM_Dashboard	Query-driven	Read-only command view with 5 sections
6	PAM_Settings	3 (A-C)	Key-value configuration store

Theme: Dark background `#0A0A0C`, gold accents `#C9A84C`, surface `#141418`, gridlines hidden.

D) Schema -- PAM_Projects (13 columns)

Col	Header	Type	Calc/Manual
A	Project ID	String	CALC (P001, P002...)
B	Project Name	String	Manual
C	Project Type	String	Dropdown
D	Category	String	Manual
E	Description	String	Manual
F	Budget	Currency	Manual
G	Spent So Far	Currency	CALC (SUMIFS from PAM_Purchases)
H	Remaining Budget	Currency	CALC (F - G)
I	Priority	String	Dropdown
J	Status	String	Dropdown
K	Start Date	Date	Manual
L	Target Completion	Date	Manual
M	Notes	String	Manual

Project Type Dropdown: Vehicle Flip, Vehicle Build, Property Rehab, ATV/Powersports Build, Solar/Off-Grid, Electronics Bundle, General Arbitrage, Other

Priority Dropdown: Critical, High, Medium, Low

Status Dropdown: Planning, Active, Paused, Complete, Cancelled

Formulas:

- G (Spent): `=SUMIFS(PAM_Purchases!F:F, PAM_Purchases!B:B, A{row})`
- H (Remaining): `=F{row}-G{row}`

Conditional Formatting: Row turns `#3A0A0A` with red font `#FF6B6B` when remaining budget < 10% of budget.

E) Schema -- PAM_Needs (19 columns)

Col	Header	Type	Calc/Manual
A	Need ID	String	CALC (N001, N002...)
B	Project ID	String	Foreign key

Col	Header	Type	Calc/Manual
C	Project Name	String	CALC (VLOOKUP from PAM_Projects)
D	Item Needed	String	Manual
E	Keyword Set	String	Manual (CSV keywords for matching)
F	Category	String	Dropdown
G	Subcategory	String	Manual
H	Preferred Brand	String	Manual
I	Preferred Model	String	Manual
J	Acceptable Alternatives	String	Manual
K	Condition Needed	String	Dropdown
L	Qty Needed	Number	Manual
M	Qty Acquired	Number	CALC (COUNTIFS from PAM_Purchases)
N	Target Price	Currency	Manual
O	Max Price	Currency	Manual
P	Priority	String	Dropdown
Q	Required	Boolean	Checkbox
R	Notes	String	Manual
S	Status	String	Dropdown

Category Dropdown (17 options): HVAC, Electrical, Plumbing, Body/Cosmetic, Drivetrain, Engine, Suspension, Wheels/Tires, Interior, Exterior, Appliance, Flooring, Solar, Battery, Electronics, Tools, Other

Condition Dropdown: New, Like New, Good, Fair, Any

Priority Dropdown: Critical, Important, Nice to Have

Status Dropdown: Open, Partially Filled, Fulfilled, Cancelled

Formulas:

- C (Project Name): `=VLOOKUP(B{row}, PAM_Projects!A:B, 2, FALSE)`
- M (Qty Acquired): `=COUNTIFS(PAM_Purchases!C:C, B{row}, PAM_Purchases!D:D, D{row})`

F) Schema -- PAM_Matches (25 columns)

Col	Header	Type	Source
A	Match ID	String	CALC (M001, M002...)
B	Scraped Row ID	String	Layer 1
C	Project ID	String	Layer 1
D	Project Name	String	CALC (VLOOKUP)
E	Need ID	String	Layer 1
F	Need Item	String	CALC (VLOOKUP)
G	Secondary Need ID	String	Layer 1 (cross-project)
H	Listing Title	String	Layer 1 (from source)
I	Listing Description	String	Layer 1 (from source)
J	Platform	String	Layer 1 (from source)

Col	Header	Type	Source
K	Listing URL	URL	Layer 1 (from source)
L	Asking Price	Currency	Layer 1 (from source)
M	Estimated Value	Currency	Layer 1
N	Distance	String	Layer 1 (from source)
O	Logic Match	Boolean	Layer 1
P	Match Score	Number 0-100	Layer 1
Q	Match Tier	String	Layer 1 (Strong/Good/Weak)
R	AI Fit Verdict	String	Layer 2
S	AI Compatibility Insight	String	Layer 2
T	AI Strategy	String	Layer 2
U	AI Risk Flags	String	Layer 2
V	AI Negotiation Angle	String	Layer 2
W	Recommended Action	String	Layer 2
X	Review Status	String	Dropdown
Y	Timestamp	DateTime	CALC

Review Status Dropdown: New, Reviewed, Acted On, Dismissed

AI Fit Verdict Values: Perfect Fit, Likely Fit, Possible Fit, Not Recommended

AI Strategy Values: Buy for Project, Buy + Flip Extra, Watch, Pass

Match Tier Classification:

- Strong: score >= 80
- Good: score >= 60
- Weak: score < 60

G) Schema -- PAM_Purchases (15 columns)

Col	Header	Type	Calc/Manual
A	Purchase ID	String	CALC (PUR001, PUR002...)
B	Project ID	String	Manual
C	Need ID	String	Manual
D	Item Bought	String	Manual
E	Source / Platform	String	Manual
F	Purchase Price	Currency	Manual
G	Date Bought	Date	Manual
H	Seller	String	Manual
I	Listing URL	URL	Manual
J	Match ID	String	Manual (links back to match)
K	Installed Yet	Boolean	Checkbox
L	Resellable Later	Boolean	Checkbox
M	Estimated Resale Value	Currency	Manual
N	Condition	String	Dropdown

Col	Header	Type	Calc/Manual
O	Notes	String	Manual

Condition Dropdown: New, Like New, Good, Fair, Poor

Side Effects: Recording a purchase auto-updates:

- Match status → "Acted On" (if Match ID provided)
- Need status → "Partially Filled" or "Fulfilled" (based on qty comparison)
- Project "Spent So Far" → recalculated via SUMIFS formula

H) Schema -- PAM_Dashboard (Query-Driven)

Read-only sheet with 5 QUERY-driven sections:

Section	Rows	Source	Content
Active Projects	3-21	PAM_Projects	WHERE Status = 'Active'
Top Unfilled Needs	22-41	PAM_Needs	WHERE Status = 'Open' AND Priority IN ('Critical','Important'), LIMIT 15
Newest Strong/Good Matches	42-61	PAM_Matches	WHERE Tier IN ('Strong','Good'), ORDER BY timestamp DESC, LIMIT 15
Recent Purchases	62-76	PAM_Purchases	ORDER BY date DESC, LIMIT 10
Budget Health	77+	PAM_Projects	Budget bars for Active projects

---

I) Schema -- PAM_Settings (3 columns)

Col	Header
A	Setting
B	Value
C	Notes

Settings Keys & Defaults

Key	Default	Type	Purpose
PAM_SourceSheet	Master DB	String	Sheet to scan for scraped listings
PAM_KeywordWeight	40	Number	Keyword match scoring weight
PAM_CategoryWeight	20	Number	Category match scoring weight
PAM_PriceWeight	20	Number	Price vs target scoring weight
PAM_BrandWeight	10	Number	Brand/model match bonus weight
PAM_DistanceWeight	5	Number	Distance scoring weight
PAM_ConditionWeight	5	Number	Condition clue scoring weight
PAM_AIThreshold	60	Number	Min score to trigger AI evaluation
PAM_AIModel	gpt-4o-mini	String	OpenAI model for evaluations

Key	Default	Type	Purpose
PAM_MaxAIPerRun	20	Number	Max AI calls per engine run
PAM_DistanceRadius	30	Number	Max distance in miles
PAM_APIKeyCell	Settings!B2	String	Cell reference for OpenAI API key

Note: Scoring weights should sum to 100. The UI validates this with a live indicator.

J) Layer 1 -- Logic Engine

File: PAM_LogicEngine.gs (228 lines)

`runPAMLogicEngine()` -- the rule-based matching engine.

How It Works

1. Loads settings (weights, source sheet, distance radius)
2. Reads source sheet with flexible column mapping (case-insensitive header detection)
3. Loads all open needs (Status = 'Open' or 'Partially Filled')
4. For each scraped listing, scores against every open need using 6 criteria:

Criterion	Weight (default)	Scoring Logic
Keyword Match	40	`pamKeywordScore_()` ratio × weight
Category Match	20	Case-insensitive substring containment
Price Score	20	Full points if asking <= target; half if <= max
Brand/Model Bonus	10	Any brand/model token (>2 chars) found in text
Distance Score	5	Full points if within radius; half if no data
Condition Score	5	Hierarchy check: New > Like New > Good > Fair > Any

5. Best match selected per listing; secondary need tracked if different project scores >= 40
6. **Minimum threshold: 40** -- listings below 40 are skipped
7. **Deduplication:** skips if same scraped row + need already matched
8. **Tier assignment:** Strong (>= 80), Good (>= 60), Weak (< 60)
9. Writes to PAM_Matches with AI columns (R-W) left empty for Layer 2

Source Column Mapping (Flexible)

Field	Accepted Headers
Title	title, listing title, name, item
Description	description, desc, details, body
Price	price, asking, ask price, cost
Category	category, type
Distance	distance, miles, mi
Platform	platform, source, site, marketplace
URL	url, link, listing url
Row ID	row id, id, row, listing id

Condition Hierarchy

```
New > Like New > Good > Fair > Any
```

Condition keyword map:

- **New:** new, brand new, unopened, sealed, oem
- **Like New:** like new, excellent, mint, pristine, barely used
- **Good:** good, great condition, works great, fully functional
- **Fair:** fair, some wear, used, functional

Returns: `{ success, log, summary, written, strong, good, weak, skipped }`

K) Layer 2 -- AI Engine

File: PAM_AIEngine.gs (258 lines)

`runPAMAIEvaluation()` -- the OpenAI evaluation engine.

How It Works

1. Loads settings (AI threshold, model, max calls per run)
2. Fetches OpenAI API key from `Settings!B2`
3. Identifies qualifying matches: score >= threshold AND no AI verdict yet (or "AI Error")
4. For each match (up to max per run):
   - Builds context prompt with project, need, listing, and secondary need info
   - Calls OpenAI with JSON mode
   - Rate limits: 1.1 second sleep between calls
   - Writes AI response to columns R-W
5. On error: writes "AI Error -- retry" (eligible for re-evaluation next run)

OpenAI API Configuration

Setting	Value
Endpoint	`https://api.openai.com/v1/chat/completions`
Default Model	gpt-4o-mini
Supported Models	gpt-4o-mini, gpt-4o, gpt-4-turbo
Temperature	0.3
Max Tokens	500
Response Format	JSON object (structured output)

AI Response JSON Schema

```
{
fit_verdict: "Perfect Fit | Likely Fit | Possible Fit | Not Recommended",
compatibility_insight: "string (why it fits or doesn't)",
strategy: "Buy for Project | Buy + Flip Extra | Watch | Pass",
risk_flags: "string (potential issues)",
negotiation_angle: "string (how to approach the seller)"
}
```

Strategy → Recommended Action Mapping

The AI strategy maps to a user-facing Recommended Action in column W.

Full Cycle

`runPAMFullCycle()` runs both engines sequentially:

1. `runPAMLogicEngine()` (Layer 1)

2. 500ms pause
3. `runPAMAIEvaluation()` (Layer 2)

Returns: `{ success, logic, ai, log }`

L) Menu Structure

Menu: PAM (top-level, registered via `addPAMMenu()`)

Item	Function
Open Command Center	`openPAMDashboard()`
-- separator --	
Run Logic Engine	`runPAMLogicEngine()`
Run AI Evaluation	`runPAMAIEvaluation()`
Full Cycle (Logic → AI)	`runPAMFullCycle()`
-- separator --	
Initialize PAM Sheets	`initializePAMSheets()`

M) Command Center UI (PAM_UI.html)

Type: Modal dialog (1400 x 900)

Lines: 1,654

External Libraries: Google Fonts (Playfair Display)

Design System

Token	Value
--bg	#0A0A0C
--surface	#141418
--surface2	#1A1A20
--gold	#C9A84C
--blue	#4080C4
--green	#40A060
--amber	#C48840
--red	#C44040
--mono	SF Mono, Cascadia Code, JetBrains Mono, Consolas
--serif	Playfair Display

Includes noise overlay (inline SVG fractal noise for texture).

Tab 1: Projects

- Stats row: Total Projects, Active, Total Budget, Total Spent
- Search filter
- Inline "New Project" form with all 13 fields
- Expandable project cards with budget bar visualization
- Linked needs shown inline when expanded
- "Add Need" button within project expansion

Tab 2: Needs

- Filter bar: text search, priority dropdown, status dropdown
- Inline "Add Need" form with all 19 fields
- Needs grouped by project name with collapsible headers
- Each need shows: ID, item, keywords excerpt, priority/status badges, qty bar, price range

Tab 3: Matches

- Stats row: Total Matches, Strong (green), Awaiting AI (amber), Acted Today (blue)
- Filter bar: Project, Tier, AI Verdict, Status dropdowns + Refresh button
- Expandable match cards showing:
- Score badge (circular, color-coded), tier, listing title, platform, project/need, distance, price
- AI verdict, review status
- Expanded: description, AI compatibility, strategy, risk flags, negotiation angle, listing URL
- Action buttons: Buy for Project (green), Buy + Flip (blue), Watch (amber), Dismiss (ghost)

Purchase Modal

- Overlay triggered from match action buttons
- Pre-populates from match data
- Fields: Item, Project, Price, Platform, Seller, Date, Condition, URL, Resale Value, Notes

Tab 4: Run Engine

- 3 action buttons: Logic Engine, AI Evaluation, Full Cycle (all with pulsing animations)
- Two-column layout:
- Left: Execution log console (terminal-style, color-coded by category)
- Right: Current settings preview + "Initialize/Repair PAM Sheets" button
- Log categories: START (gold), SCAN (blue), EVAL (amber), OK (green), DONE (green bold), ERROR (red), WARN (amber), INFO (muted)

Tab 5: Settings

- Data Source: source sheet name, API key cell reference
- Scoring Weights: 6 number inputs with live sum validation (green at 100, red otherwise)
- AI Evaluation: threshold slider with live tier preview, model dropdown, max calls, distance radius
- Save Settings / Reset to Defaults buttons

Server Calls (google.script.run)

Call	Purpose
`getPAMProjects()`	Fetch all projects
`getPAMNextIds()`	Get next auto-increment IDs
`getPAMNeeds(projectId)`	Fetch needs (optionally by project)
`getPAMMatches(null)`	Fetch all matches
`getPAMStats()`	Fetch aggregate match statistics
`getPAMSettings()`	Fetch current settings
`addProject(data)`	Create a new project
`addNeed(data)`	Create a new need
`recordPurchase(data)`	Record a purchase
`updateMatchStatus(id, status)`	Update match review status

Call	Purpose
`runPAMLogicEngine()`	Execute Layer 1
`runPAMAIEvaluation()`	Execute Layer 2
`runPAMFullCycle()`	Execute both layers
`savePAMSettings(data)`	Persist settings
`initializePAMSheets()`	Create/repair sheets

N) Test Data Seeder

`seedPAMTestData()` creates sample data for demonstration:

3 Test Projects:

Name	Type	Budget
Berkeley House Rebuild	Property Rehab	$15,000
Yamaha Blaster Build	ATV/Powersports Build	$2,500
CarHawk Flip 04 -- Mustang	Vehicle Flip	$4,000

7 Test Needs:

Project	Item	Priority	Target	Max	Keywords
Berkeley House	Mini Split AC Unit	Critical	$250	$500	mini split, ductless, heat pump, condenser, air handler, mr slim, mrcool, mitsubishi mini split
Berkeley House	Exterior Entry Door	Important	$100	$250	entry door, exterior door, front door, steel door, fiberglass door, prehung
Berkeley House	Bathroom Vanity	Nice to Have	$75	$200	bathroom vanity, vanity cabinet, sink vanity, bath cabinet
Yamaha Blaster	Front Plastics Set	Critical	$40	$80	blaster plastics, front fender, blaster fender, yfs200 plastics, blaster body
Yamaha Blaster	Brake Setup	Important	$30	$60	blaster brakes, brake caliper, brake pads, yfs200 brakes, rear brake
Mustang Flip	Headlight Assembly	Critical	$80	$150	mustang headlight, headlamp, headlight assembly, ford headlight
Mustang Flip	Front Seat Set	Important	$150	$300	mustang seats, front seats, bucket seats, ford seats, leather seats

---

O) Function Index

File	Function	Visibility
PAM_Settings.gs	`getPAMSettingsSheet_()`	Private
PAM_Settings.gs	`getPAMSettings()`	Public
PAM_Settings.gs	`savePAMSettings(data)`	Public
PAM_Settings.gs	`getPAMSetting_(key)`	Private
PAM_Settings.gs	`getOpenAIKey_()`	Private
PAM_Settings.gs	`initPAMSettings()`	Public
PAM_Utils.gs	`pamNextId_(sheet, prefix, col)`	Private
PAM_Utils.gs	`pamNextProjectId()`	Public
PAM_Utils.gs	`pamNextNeedId()`	Public
PAM_Utils.gs	`pamNextMatchId()`	Public
PAM_Utils.gs	`pamNextPurchaseId()`	Public
PAM_Utils.gs	`pamMatchExists_(scrapedRowId, needId)`	Private
PAM_Utils.gs	`pamFormatCurrency_(val)`	Private
PAM_Utils.gs	`pamFormatDate_(val)`	Private
PAM_Utils.gs	`pamSafeStr_(v)`	Private
PAM_Utils.gs	`pamKeywordScore_(text, keywordCsv)`	Private
PAM_Utils.gs	`pamContainsCondition_(text, conditionNeeded)`	Private
PAM_Utils.gs	`pamStyleHeader_(sheet, numCols)`	Private
PAM_Utils.gs	`pamApplyDropdown_(sheet, row, col, options)`	Private
PAM_Utils.gs	`pamApplyDropdownColumn_(sheet, col, options, startRow, endRow)`	Private
PAM_Utils.gs	`pamLog_(message, category)`	Private
PAM_Utils.gs	`pamGetLog()`	Public
PAM_Utils.gs	`pamParseJSON_(str)`	Private
PAM_Sheets.gs	`initializePAMSheets()`	Public
PAM_Sheets.gs	`createPAMProjectsSheet_()`	Private
PAM_Sheets.gs	`createPAMNeedsSheet_()`	Private
PAM_Sheets.gs	`createPAMMatchesSheet_()`	Private
PAM_Sheets.gs	`createPAMPurchasesSheet_()`	Private
PAM_Sheets.gs	`createPAMDashboardSheet_()`	Private
PAM_DataAccess.gs	`getPAMProjects()`	Public
PAM_DataAccess.gs	`addProject(data)`	Public
PAM_DataAccess.gs	`getPAMNeeds(projectId)`	Public
PAM_DataAccess.gs	`addNeed(data)`	Public
PAM_DataAccess.gs	`getPAMMatches(filters)`	Public
PAM_DataAccess.gs	`updateMatchStatus(matchId, status)`	Public
PAM_DataAccess.gs	`recordPurchase(data)`	Public
PAM_DataAccess.gs	`refreshNeedStatus_(needId)`	Private
PAM_DataAccess.gs	`getPAMStats()`	Public

File	Function	Visibility
PAM_DataAccess.gs	`getPAMNextIds()`	Public
PAM_LogicEngine.gs	`runPAMLogicEngine()`	Public
PAM_AIEngine.gs	`runPAMAIEvaluation()`	Public
PAM_AIEngine.gs	`buildAIPrompt_(...)`	Private
PAM_AIEngine.gs	`callOpenAI_(apiKey, model, prompt)`	Private
PAM_AIEngine.gs	`buildNeedsMap_()`	Private
PAM_AIEngine.gs	`buildProjectsMap_()`	Private
PAM_AIEngine.gs	`runPAMFullCycle()`	Public
PAM_Menu.gs	`addPAMMenu()`	Public
PAM_Menu.gs	`openPAMDashboard()`	Public
PAM_Menu.gs	`seedPAMTestData()`	Public

Total: 49 functions across 7 .gs files + 29 client-side JS functions in PAM_UI.html

---

P) Implementation Status

All items are **COMPLETE** [x]:

- [x] PAM_Settings.gs -- 12 configurable settings with defaults
- [x] PAM_Utils.gs -- ID generation, keyword scoring, condition hierarchy, formatting
- [x] PAM_Sheets.gs -- 6 sheets with schemas, formulas, dropdowns, conditional formatting
- [x] PAM_DataAccess.gs -- Full CRUD for all sheets, stats aggregation, status tracking
- [x] PAM_LogicEngine.gs -- 6-criterion weighted scoring with flexible column mapping
- [x] PAM_AIEngine.gs -- OpenAI gpt-4o-mini evaluation with JSON structured output
- [x] PAM_Menu.gs -- Menu registration, dashboard launcher, test data seeder
- [x] PAM_UI.html -- 5-tab Command Center with full CRUD, engine controls, settings

---

Updated Spec Pack Index

Part	File	Covers
1	SPEC_PACK_PART1_SYSTEM_OVERVIEW.md	Identity, architecture, file inventory, config, menu, initialization
2	SPEC_PACK_PART2_SHEET_SCHEMAS.md	25 core + Turo sheet schemas
3	SPEC_PACK_PART3_AI_ENGINE.md	OpenAI integration, calculations, scoring, knowledge base
4	SPEC_PACK_PART4_CRM_ENGINE.md	CRM pipeline, follow-ups, SMS/email, automation, API layer
5	SPEC_PACK_PART5_BROWSE_AI.md	Browse.AI robots, import pipeline, 10 platforms, URL builders
6	SPEC_PACK_PART6_INTEGRATIONS.md	SMS-iT, Ohmylead, CompanyHub, Twilio, SendGrid, credentials
7	SPEC_PACK_PART7_UI_COMPONENTS.md	18 HTML dialogs, server-side HTML, design system
8	SPEC_PACK_PART8_TRIGGERS_ALERTS.md	Triggers, alerts, reporting, dashboard, testing, utilities

Part	File	Covers
9	**SPEC_PACK_PART9_PAM.md**	**Project Arbitrage Module -- projects, needs, matching, AI evaluation**
Turo	TURO_SPEC_PACK.md	Complete Turo Rental Hold Module specification

CarHawk Ultimate — Turo Rental Hold Add-On Module

Spec Pack v1.0.0

A) Module Overview

The Turo Module extends CarHawk Ultimate (Quantum-2.0.0) with rental fleet analysis capabilities. It enables dealers and car flippers to evaluate whether a vehicle should be held for Turo rental income rather than flipped immediately, providing a data-driven Flip vs. Hold decision framework.

Architecture Role: Additive module -- zero breaking changes to existing CarHawk core. All Turo logic lives in 7 dedicated `.gs` files plus minor integration hooks in `main.gs`. The module reads from the existing Master Database and writes results back to 10 new appended columns.

Workflow Integration:

1. Import deals via existing Quantum pipeline -> Master Database
2. Run Turo analysis (single or batch) from the menu
3. Turo Engine computes economics, scores, and recommendations
4. Results write to both Turo Engine sheet and Master Database writeback columns
5. Qualifying vehicles can be added to the Fleet Manager for ongoing tracking
6. Maintenance events and compliance alerts tracked separately
7. Nightly trigger auto-runs batch analysis and compliance checks

B) Sheet Inventory

#	Sheet Name	Columns	Tab Color	Purpose
1	Turo Engine	39 (A-AM)	#00BCD4 (Teal)	Core analysis -- economics, scoring, recommendations
2	Fleet Manager	26 (A-Z)	#4CAF50 (Green)	Active fleet tracking, financials, ROI
3	Maintenance & Turnovers	14 (A-N)	#FF9800 (Orange)	Service logging, cost tracking, downtime
4	Turo Pricing & Seasonality	Matrix	#9C27B0 (Purple)	Base rates by class + seasonal multipliers
5	Insurance & Compliance	17 (A-Q)	#F44336 (Red)	Registration, insurance, inspection tracking

C) Schema -- Exact Headers Per Sheet

Turo Engine (39 columns)

Col	Header	Type	Calc/Manual
A	Row ID	String	CALC (MD-{row})
B	VIN	String	COPY from MD
C	Vehicle	String	CALC (Year Make Model Trim)

Col	Header	Type	Calc/Manual
D	Vehicle Class	String	CALC (classifyVehicle)
E	Asking Price	Currency	COPY from MD
F	Estimated Repair Cost	Currency	COPY from MD
G	Total Acquisition Cost	Currency	CALC (E+F)
H	Estimated Resale Value	Currency	COPY from MD
I	Flip Net Profit	Currency	COPY from MD
J	Flip Timeline (days)	Integer	LOOKUP from pricing
K	Daily Rate	Currency	LOOKUP/OVERRIDE
L	Utilization %	Percent	LOOKUP/OVERRIDE
M	Gross Monthly Revenue	Currency	CALC (K x 30.44 x L)
N	Turo Platform Fee %	Percent	SETTINGS
O	Turo Platform Fee $	Currency	CALC (M x N)
P	Insurance Monthly	Currency	LOOKUP/OVERRIDE
Q	Cleaning Per Trip	Currency	SETTINGS
R	Trips Per Month	Decimal	CALC
S	Total Cleaning Monthly	Currency	CALC (Q x R)
T	Maintenance Reserve Monthly	Currency	CALC
U	Depreciation Monthly	Currency	CALC
V	Financing Payment Monthly	Currency	CALC (PMT)
W	Registration & Tax Monthly	Currency	CALC
X	Total Monthly Expenses	Currency	CALC (sum O-W)
Y	Net Monthly Cash Flow	Currency	CALC (M - X)
Z	Net Annual Cash Flow	Currency	CALC (Y x 12)
AA	Payback Months	Decimal	CALC (G / Y)
AB	Break-Even Utilization %	Percent	CALC
AC	12-Month Total Profit (Turo)	Currency	CALC
AD	12-Month Total Profit (Flip)	Currency	COPY
AE	Turo vs Flip Delta	Currency	CALC (AC - AD)
AF	Turo Hold Score	Integer	CALC (0-100)
AG	Risk Tier	String	CALC
AH	Recommended Strategy	String	CALC
AI	Rationale	String	CALC
AJ	Exit Plan	String	CALC
AK	Turo Status	String	MANUAL/CALC
AL	Date Evaluated	Date	CALC (now)
AM	Override?	Boolean	MANUAL

Fleet Manager (26 columns)

Col	Header	Type
A	Fleet ID	String (TF-NNN)

Col	Header	Type
B	VIN	String
C	Vehicle	String
D	Vehicle Class	String
E	Turo Status	Dropdown
F	Date Acquired	Date
G	Date Listed on Turo	Date
H	Acquisition Cost	Currency
I	Total Revenue to Date	Currency
J	Total Expenses to Date	Currency
K	Net Profit to Date	Currency (CALC)
L	ROI to Date %	Percent (CALC)
M	Months Active	Integer (CALC)
N	Avg Monthly Revenue	Currency (CALC)
O	Avg Monthly Net	Currency (CALC)
P	Utilization to Date %	Percent
Q	Trips to Date	Integer
R	Avg Trip Length (days)	Decimal
S	Current Daily Rate	Currency
T	Monthly Insurance	Currency
U	Last Service Date	Date
V	Next Service Due	Date
W	Current Estimated Value	Currency (CALC)
X	Projected Payoff Date	Date (CALC)
Y	On Track?	String (CALC)
Z	Fleet Notes	String

Maintenance & Turnovers (14 columns)

Col	Header	Type
A	Log ID	String (MT-NNN)
B	Fleet ID	String
C	Vehicle	String
D	Date	Date
E	Type	Dropdown
F	Description	String
G	Cost	Currency
H	Vendor	String
I	Mileage at Service	Integer
J	Downtime Days	Integer
K	Next Service Type	Dropdown
L	Next Service Date	Date

Col	Header	Type
M	Next Service Mileage	Integer
N	Notes	String

Rows 2-5 contain summary formulas (Total Spend, Total Downtime, Avg Cost, Avg Downtime).

Insurance & Compliance (17 columns)

Col	Header	Type
A	Fleet ID	String
B	Vehicle	String
C	Registration State	String
D	Registration Expiry	Date
E	Registration Cost	Currency
F	Insurance Provider	String
G	Insurance Policy #	String
H	Insurance Expiry	Date
I	Insurance Monthly Premium	Currency
J	Insurance Type	Dropdown
K	Inspection Due	Date
L	Inspection Status	Dropdown
M	Emissions Due	Date
N	Annual Property Tax	Currency
O	LLC/Business Entity	String
P	Title Status	Dropdown
Q	Compliance Notes	String

D) New Master Database Columns (BJ-BS)

Col	Header	Type	Source
BJ	Turo Hold Score	Integer (0-100)	CALC by Turo Engine
BK	Turo Monthly Net	Currency	CALC by Turo Engine
BL	Turo Payback Months	Decimal	CALC by Turo Engine
BM	Turo Break-Even Util %	Percent	CALC by Turo Engine
BN	Turo Risk Tier	String	CALC by Turo Engine
BO	Turo vs Flip Delta	Currency	CALC by Turo Engine
BP	Turo Recommended?	String	CALC by Turo Engine
BQ	Turo Status	String	CALC/MANUAL
BR	Fleet ID	String	Set by addToFleet()
BS	Turo Notes	String	MANUAL

E) Turo Engine -- Formulas & Logic

Gross Monthly Revenue

```
grossMonthlyRevenue = dailyRate x 30.44 x utilization
```

Platform Fee

```
platformFeeAmt = grossMonthlyRevenue x platformFeePct (default 25%)
```

Total Monthly Expenses

```
totalMonthlyExpenses = platformFee + insurance + cleaning + maintenance + depreciation + financing
+ registration
```

Net Monthly Cash Flow

```
netMonthlyCashFlow = grossMonthlyRevenue - totalMonthlyExpenses
```

Payback Months

```
paybackMonths = totalAcquisitionCost / netMonthlyCashFlow (Infinity if negative)
```

Break-Even Utilization

```
breakEvenUtilization = fixedMonthlyExpenses / (dailyRate x 30.44 x (1 - platformFeePct) -
cleaningPerMonth)
```

12-Month Comparison

```
turo12MonthProfit = (netMonthlyCashFlow x 12) + depreciatedResale12 - totalAcquisitionCost
flip12MonthProfit = flipNetProfit (one-time)
turoVsFlipDelta = turo12MonthProfit - flip12MonthProfit
```

Turo Hold Score (0-100)

Weighted composite of 6 sub-scores:

Component	Weight	Sub-Score Formula		
Payback	0.25	max(0, 100 - paybackMonths x 4)		
Cash Flow	0.25	min(100, netMonthlyCashFlow / 5)		
Utilization	0.15	utilization x 100		
Depreciation	0.15	max(0, 100 - annualDepRate x 500)		
Flip Comparison	0.10	100 if delta > 0, else max(0, 100 -	delta	/20)
Mileage	0.10	max(0, 100 - mileage/2000)		

Risk Tiers

Score Range	Tier	Description
70-100	Low	Strong Turo candidate
50-69	Medium	Viable but monitor
30-49	High	Consider flipping
0-29	Critical	Do not Turo

Flip Strategy Override Rules

- Score >= 70 AND delta > 0: Set Flip Strategy to "Turo Hold"

- Score >= 50 AND delta > -500: Keep existing strategy (marginal)
- Below 50: Keep existing strategy (flip is better)
- Override? checkbox prevents any automatic changes

Turo Recommended? Labels

- "YES -- Turo Hold": score >= 70 AND delta > 0
- "MARGINAL": score >= 50 AND delta > -500
- "INSUFFICIENT DATA": missing price or resale data
- "NO -- Flip Better": all other cases

F) Vehicle Classification Logic

Priority order (first match wins):

1. **Luxury**: Make in LUXURY_BRANDS list (24 brands)
2. **Sports/Exotic**: Model matches SPORTS_MODELS list (50+ models)
3. **Truck**: Model matches TRUCK_MODELS list
4. **SUV**: Model matches SUV_MODELS list
5. **Van/Minivan**: Model matches VAN_MODELS list
6. **Economy**: Model matches ECONOMY_MODELS list
7. **Full-Size**: Model matches FULL_SIZE_MODELS list
8. **Midsize**: Model matches MIDSIZE_MODELS list
9. **Body Type Fallback**: truck/pickup -> Truck, suv/crossover -> SUV, van/minivan -> Van/Minivan, hatchback -> Economy
10. **Default**: Midsize

G) Fleet Lifecycle Model

```
Candidate -> Acquired -> Listed -> Active -> In Maintenance
| | | |
| | +-> Paused +-> Paused
| | | |
| +--------+-> Retired +-> Active
| +-> Retired
+-> Retired
Retired -> Sold (terminal)
```

Valid transitions:

From	Valid Next States
Candidate	Acquired
Acquired	Listed, Retired
Listed	Active, Paused, Retired
Active	In Maintenance, Paused, Retired
In Maintenance	Active, Paused, Retired
Paused	Active, Listed, Retired
Retired	Sold
Sold	(terminal)

H) Settings Keys & Defaults

Key	Default	Description
TURO_PLATFORM_FEE_PCT	0.25	Turo's host cut (25% for Go plan)
TURO_PROTECTION_PLAN	Go	Go (25%), Standard (15%), Premium (10%)
TURO_AVG_TRIP_LENGTH	3	Average trip length in days
TURO_CLEANING_PER_TRIP	30	Cleaning cost between renters
TURO_MAINTENANCE_RESERVE_PCT	0.06	Annual maintenance reserve %
TURO_DEPRECIATION_MODEL	Tiered	Tiered (by age), Straight-Line, or None
TURO_RENTAL_DEPRECIATION_ADD	0.02	Additional annual dep from rental wear
TURO_FINANCING_APR	0	0 = cash purchase
TURO_FINANCING_TERM	0	0 = cash, else months
TURO_ANNUAL_REGISTRATION	275	Average annual registration cost
TURO_ANNUAL_INSPECTION	50	State inspection cost
TURO_USE_SEASONAL	false	Apply monthly seasonal multipliers
TURO_MIN_SCORE	50	Minimum acceptable Turo score
TURO_WEIGHT_PAYBACK	0.25	Score weight: payback
TURO_WEIGHT_CASHFLOW	0.25	Score weight: cash flow
TURO_WEIGHT_UTILIZATION	0.15	Score weight: utilization
TURO_WEIGHT_DEPRECIATION	0.15	Score weight: depreciation
TURO_WEIGHT_FLIP_COMPARISON	0.10	Score weight: flip comparison
TURO_WEIGHT_MILEAGE	0.10	Score weight: mileage
TURO_MODULE_ENABLED	true	Master toggle for Turo module

I) Turo Pricing & Seasonality Defaults

Base Rates by Vehicle Class

Class	Daily Rate	Util %	Trip Len	Insurance/ mo	Reg/yr	Clean/trip	Maint %	Flip Days
Economy	$35	65%	3	$150	$200	$25	5%	14
Midsize	$50	60%	3	$175	$250	$30	5%	21
Full-Size	$60	55%	4	$200	$300	$35	6%	21
SUV	$70	58%	4	$225	$350	$40	6%	28
Truck	$65	52%	3	$200	$300	$40	7%	21
Luxury	$120	45%	2	$350	$500	$60	8%	35
Sports/Exotic	$175	35%	2	$500	$600	$60	10%	45
Van/Minivan	$55	50%	5	$175	$275	$35	6%	30

Seasonal Multipliers

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Rate	0.80	0.85	0.90	1.00	1.10	1.20	1.25	1.20	1.00	0.90	0.85	0.95
Util	0.75	0.80	0.90	1.00	1.10	1.15	1.20	1.15	1.00	0.85	0.80	0.90

J) Menu Commands

#	Menu Item	Function
1	Analyze Selected Deal for Turo	`analyzeTuroSelected`
2	Batch Analyze All Turo Candidates	`batchAnalyzeTuro`
3	Refresh Fleet Dashboard	`refreshFleetManager`
4	Add Vehicle to Fleet	`addToFleetSelected`
5	Update Fleet Financials	`updateFleetFinancials`
6	Log Maintenance Event	`logMaintenanceEvent`
7	Check Compliance Alerts	`checkComplianceAlerts`
8	Setup Turo Module	`setupTuroModule`

All items are under the "Turo Module" submenu within the main "CarHawk Ultimate" menu.

K) Triggers & Automation

Nightly Trigger Extension

Added to `quantumDailyAnalysis()`:

- `batchAnalyzeTuro()` -- Re-analyzes all qualifying candidates
- `checkComplianceAlerts()` -- Scans for expiring registrations, insurance, inspections

Only runs if the Turo Engine sheet exists (module installed check).

L) CRM Field Mapping

When Flip Strategy = "Turo Hold", the CompanyHub export includes:

CRM Field	Source
turoHoldScore	Master DB: Turo Hold Score
turoMonthlyNet	Master DB: Turo Monthly Net
turoPaybackMonths	Master DB: Turo Payback Months
turoRiskTier	Master DB: Turo Risk Tier
turoStatus	Master DB: Turo Status
fleetId	Master DB: Fleet ID

Plus all existing 16 CompanyHub fields (Company, Contact, Phone, Email, Deal Name, Deal Value, Expected Profit, ROI%, Stage, Probability, Expected Close Date, Lead Score, Source, Location, Distance, Days on Market, Contact Attempts, Response Rate, Tags).

M) GAS File Inventory

File	Lines	Purpose
`turo_config.gs`	563	Constants, vehicle classification maps, column indices, picklists, defaults
`turo_setup.gs`	603	Idempotent sheet creation, MD column append, settings injection
`turo_engine.gs`	872	Core analysis: classification, economics, scoring, writeback
`turo_fleet.gs`	448	Fleet management: add vehicles, refresh financials, lifecycle
`turo_maintenance.gs`	239	Maintenance event logging with HTML dialog
`turo_compliance.gs`	235	Insurance/compliance alerts, email notifications
`turo_tests.gs`	565	7 acceptance tests with master runner
Total	**3,525**	

Modified existing file:

- `main.gs` -- Added Turo submenu (9 items), nightly trigger extension, CRM export extension

---

N) Implementation Status

- [x] Configuration constants and vehicle classification maps (`turo_config.gs`)
- [x] Idempotent sheet setup with headers, formatting, validation (`turo_setup.gs`)
- [x] Vehicle classification algorithm (8 classes, 300+ models) (`turo_engine.gs`)
- [x] Turo economics computation with full expense breakdown (`turo_engine.gs`)
- [x] Turo Hold Score with 6 weighted sub-scores (`turo_engine.gs`)
- [x] Risk tier classification (Low/Medium/High/Critical) (`turo_engine.gs`)
- [x] Recommended Strategy with override protection (`turo_engine.gs`)
- [x] Rationale and Exit Plan generation (`turo_engine.gs`)
- [x] Single-row and batch analysis (`turo_engine.gs`)
- [x] Master Database writeback (10 columns) (`turo_engine.gs`)
- [x] Fleet ID generation (TF-NNN) (`turo_fleet.gs`)
- [x] Add to Fleet with Insurance & Compliance row creation (`turo_fleet.gs`)
- [x] Fleet Manager refresh with ROI, On Track?, depreciation (`turo_fleet.gs`)
- [x] Status lifecycle validation (`turo_fleet.gs`)
- [x] Maintenance event HTML dialog with dropdowns (`turo_maintenance.gs`)
- [x] Compliance alert scanning (60-day insurance/reg, 30-day inspection) (`turo_compliance.gs`)
- [x] Compliance email notifications (`turo_compliance.gs`)
- [x] Menu integration (8 items under Turo Module submenu) (`main.gs`)
- [x] Nightly trigger extension (`main.gs`)
- [x] CRM export Turo field enrichment (`main.gs`)
- [x] Settings injection (20 configurable settings) (`turo_setup.gs`)
- [x] Conditional formatting (Risk Tier, Turo Status, On Track?) (`turo_setup.gs`)
- [x] Data validation dropdowns (status, maintenance types, insurance types) (`turo_setup.gs`)
- [x] Acceptance test suite (7 tests) (`turo_tests.gs`)
- [x] Structured logging to Activity Logs sheet (all modules)
- [x] LockService for concurrent access protection (all write operations)
- [x] TURO_SPEC_PACK.md documentation

O) Acceptance Test Results

Tests are designed to run in a live Google Sheets environment via the Apps Script editor.

Run `runAllTuroTests()` from the script editor to execute all 7 tests:

#	Test	What It Validates
1	testSetupIdempotent	Setup runs twice without duplicating sheets/columns/settings
2	testVehicleClassification	8 vehicle classification cases (Luxury, Midsize, Economy, Truck, SUV, Sports, Van, Full-Size)
3	testTuroEngineAnalysis	Full analysis pipeline: economics, score, tier, rationale, writeback
4	testFleetCreation	Fleet ID generation, Fleet Manager row, Insurance row, MD writeback
5	testMasterDbIntegrity	Existing columns unchanged, Turo columns appended correctly, no duplicates
6	testScoreWeights	Score weights sum to 1.0
7	testLifecycleValidation	6 status transition cases (valid and invalid)

P) Remaining Integration Work

For Full CRM Sync

- CompanyHub API integration for real-time Turo field push (currently CSV export)
- SMS-iT template for Turo Hold notification to buyers

For Live Spreadsheet Testing

- Run `setupTuroModule()` on a production spreadsheet with real deal data
- Run `runAllTuroTests()` to validate in the GAS runtime environment
- Verify conditional formatting renders correctly in Google Sheets
- Test the Maintenance Event HTML dialog in the Sheets UI
- Verify email delivery for compliance alerts

Future Enhancements

- Turo API integration for real revenue/booking data import
- Automated daily rate adjustment based on market data
- Fleet performance dashboards with charts
- Multi-market pricing support (different cities)
- Integration with fleet insurance providers for automated renewal tracking